

R002 — Figure atlas: multiple shooting with guess propagation for sparse model selection

Concept figures, fresh simulations supporting the theory, solver/optimiser/parameter variations,
and the repository defects that were found and fixed

Prepared for Manu Jayadharan

report folder `reports/2026-09-11-R002-multishooting-figure-atlas/`

11 September 2026

Abstract

This atlas collects, in presentation order, every figure produced for the `hiddenSelect` project (Julia; multiple shooting with guess propagation for discovering sparse polynomial governing equations from noisy trajectories; theory manuscript *Sparse Optimization using Multi-Shooting and Guess Propagation*). It contains: (i) seven concept figures that replace the manuscript’s hand-drawn sketches (the inverse problem, single shooting, the shooting partition, multiple shooting, node removal, the guess-propagation algorithm, and the error-growth picture); (ii) numerical checks of Lemma 1, Lemma 2 and Proposition 1 of the manuscript on the FitzHugh–Nagumo (FHN) system; (iii) cost-landscape figures showing how the number of local minima, the Hessian spread and the blow-up region grow with the window size; (iv) the headline experiment — guess propagation versus restarting from the seed — over 8 optimiser seeds with paired bootstrap confidence intervals, plus variations of the window-size schedule, blow-up penalty, noise level, sparsity weight, optimiser, iteration budget, integrator sub-steps and start distance; (v) an integrator/solver comparison; (vi) the quantification of the defects reported in the code review R001 (F1, F2, F4, F5) and of two *new* defects found here (a mixed monomial ordering in the current driver and a global-variable shadowing in the single-shooting loss), all fixed in the live tree with this report; (vii) the same method on Lotka–Volterra and Lorenz. Every number in every caption is computed by scripts in this folder from a frozen code snapshot and asserted by 18 gates (§12); six MP4 animations and a slide deck are shipped alongside. Headline: with guess propagation the median parameter error over seeds falls from 1.65 (seed) to 0.32 at full single shooting, while the same optimiser without propagation stays at the seed (1.65) — paired difference -1.14 , 95 % bootstrap CI $[-1.43, -0.77]$, better in 7 of 8 seeds (the eighth seed starts on the blow-up plateau, where nothing can move; a graded penalty fixes that).

Table of figures

Figure	What is plotted	What it demonstrates / how to use it on a slide
<i>A. Concept figures (replace the hand-drawn sketches of the manuscript)</i>		
1	FHN trajectory, noisy samples, phase portrait, the 2×10 sparse coefficient matrix	The inverse problem: recover the few non-zero library coefficients from noisy samples. Opening slide.
2	Single-shooting model trajectory from y_0 at a wrong p , residual lines, $ \text{residual} $ vs t	Why single shooting is hard: the initial-condition error is amplified like e^{Lt} over the whole record.
3	Timeline of data times t_i , shooting nodes τ_k , windows, Δt , ΔT ; the coarser partition below	The shooting partition and what “removing a node” means (replaces <code>shooting_partition.png</code>).
4	Single vs multiple shooting ($\kappa = 100, 10, 5$) at the same wrong p , per-window segments, residuals	Restarting at the nodes caps the error at $e^{L\Delta T}$ (replaces <code>multiple_shooting.png</code>).

Figure	What is plotted	What it demonstrates / how to use it on a slide
5	Fine vs coarse partition around a removed node, ΔT_1 , ΔT_2 , the bound of Prop. 1	Node removal: what changes in the cost when one node is dropped (replaces <code>comparison_shooting_nodes.png</code>).
6	1-D cost slices for $\kappa = 1, 5, 25, 100$ with the minimiser carried forward; pseudo-code	The guess-propagation algorithm: solve the smooth problem first, carry its minimiser into the rugged ones.
7	Schematic error envelopes $e^{L^t}\ \eta\ $ vs the saw-tooth capped at $e^{L\Delta T}\ \eta\ $	The one-line argument for multiple shooting (uses the report's L and $\ \eta\ _{\max}$).
<i>B. Numerical checks of the theory</i>		
8	$\ \varphi(t; x_1) - \varphi(t; x_2)\ /\ x_1 - x_2\ $ for 24 probes vs $e^{\pm Lt}$, $e^{\mu t}$	Lemma 1 holds; the Lipschitz bound is very loose on the FHN limit cycle (actual amplification ≤ 8).
9	$\ \varphi(t; p_1) - \varphi(t; p_2)\ /\ p_1 - p_2\ $ vs the Lemma 2 bound	Lemma 2 holds; parameter sensitivity saturates instead of growing exponentially.
10	$ \hat{J}_K - J_K $ vs number of removed nodes and vs ΔT_2	Proposition 1 qualitatively: the cost change grows with $ I_R $ and with ΔT_2 ; remove few nodes at a time.
<i>C. Cost landscapes</i>		
11	2-D slices of J_κ in the (v, v^3) coefficient plane of $\dot{v}$, $\kappa = 1, 3, 10, 100$	The basin narrows and a blow-up plateau appears as κ grows; a positive cubic coefficient blows up in finite time.
12	Same in the $(w$ in $\dot{v}$, v in $\dot{w})$ plane	The slow $\dot{w}$ equation is weakly identifiable at small κ and better at large κ .
13	1-D slices along random directions; local-minima count vs κ	Local minima per slice grow from 1.3 ($\kappa = 1$) to 7.9 ($\kappa = 100$).
14	Hessian $\lambda_{\max}$, $ \lambda_{\min} $, $\ \nabla J_\kappa(p^*)\ $, number of negative eigenvalues vs κ ; sensitivity to x_0	Conditioning worsens by $10^{4.5}$ and the noisy cost becomes indefinite at p^* for $\kappa \geq 2$.
<i>D. Guess propagation: the headline experiment (8 seeds)</i>		
15	J_κ , $\ p - p^*\ $ and recovery score vs κ for GP, best-guess GP and no propagation	The claim of the paper, with seeds shown: GP reaches single shooting with $\ p - p^*\ \approx 0.32$; the control never leaves the seed.
16	Paired differences GP – control with bootstrap CIs; cellwise counts	The difference is decisive at every $\kappa \geq 2$ (7 of 8 seeds; CI excludes 0).
17	Seed-median coefficient error heat-map along the sweep	Which coefficients are recovered when; the w coefficient of $\dot{v}$ is shrunk by the sparsity term.
18	Filmstrip of single-shooting fits from the minimiser at $\kappa = 1, 5, 25, 100$, GP vs control (seed 2)	Visual companion of 15; still frames of <code>anim_sweep.gif</code> .
19	Nelder–Mead cost traces per κ (seed 2), GP vs control	GP starts every stage near its optimum; the control stalls on the plateau.
20	Final coefficients at $\kappa = 100$ vs truth, all seeds	Correctly labelled coefficient bar chart (the repository's labels were stale).
<i>E. Variations</i>		
21	Window-size schedule: dense (44 steps), full (16), coarse (4), jump (2)	Removing few nodes per step matters: jump 0.76 vs dense 0.30 median error.
22	Flat vs graded blow-up penalty	The graded penalty removes the “stuck on the plateau” failure (0 vs 12.5 % of seeds).
23	Noise level $\sigma_{\text{rel}} \in \{0, \dots, 0.2\}$	GP is robust up to 10 % noise and fails at 20 %; the control fails even at zero noise.
24	Sparsity weight γ	$\gamma = 0.05$ is best; $\gamma = 0$ and $\gamma = 1$ both hurt; which coefficients survive.
25	Nelder–Mead vs BFGS vs L-BFGS (AD gradient through the windowed loss)	With the F4 fix, BFGS+GP is the best and cheapest arm ($\ p - p^*\ = 0.21$, score 0.95); gradients die on the plateau.
26	Basin of attraction of one window size vs start distance	Small windows have wide basins but a noise-limited floor; single shooting has a narrow basin.
27	Nelder–Mead iteration budget; integrator sub-steps S	2500 iterations are needed; $S \geq 5$ suffices; $S = 1$ biases the fit.

Figure	What is plotted	What it demonstrates / how to use it on a slide
28	Start scale 0.01–0.3, flat vs graded penalty	Farther starts are harder; the graded penalty rescues starts that begin on the plateau.
<i>F. Integrators and solvers inside the loss</i>		
29	Accuracy vs cost per loss evaluation for 14 integrator settings	The in-house fixed-step Tsit5 ($S = 10$) matches a 10^{-12} reference to 10^{-13} and is the cheapest accurate choice; the problem is not stiff.
30	GP sweep with three integrators inside the loss	The integrator does not change the outcome; the plateau start (seed 1) does.
<i>G. Defects found and fixed</i>		
31	F2: share of the loss dropped by the mod 7 stiff branch; 1-D slice through the $\dot{w}$ equation	9–17% of $J(p^*)$ and up to 78% away from p^* was silently discarded; fixed.
32	F1: Jacobian entry, eigenvalues and trace from the truncated vs fixed RHS	The stability movies used $\partial\dot{v}/\partial v = +1.0$ instead of -1.25 ; fixed.
33	F4: cosine between the gradient the sweep received and the right one; BFGS with each	Near-orthogonal at $\kappa = 1$; BFGS stalls with the wrong gradient; fixed.
34	New: mixed monomial order in the mod 8 driver; global-data shadowing	A spurious $\ p - p^*\ = 2.50$ and an ignored argument; both fixed.
<i>H. Other systems and closing the loop</i>		
35	Lotka–Volterra (12 coefficients)	GP recovers the system ($\ p - p^*\ = 0.29$, score 0.83); the control blows up in 5 of 6 seeds.
36	Lorenz (30 coefficients, chaotic)	GP+BFGS recovers Lorenz (score 0.83); GP+Nelder–Mead does not converge; the control fails in 4 of 4 seeds.
37	Hessian at the GP minimisers; cost along the segment between consecutive minimisers; cross-evaluation	The strong-convexity premise is not verified at Nelder–Mead’s returned iterates (positive definite in 36% of cells); the carried guess sits inside the next basin.

Animations (MP4, H.264; frames regenerable by `figures/make_animations.py`; the GIF `anim_sweep.gif` is the same content as the first one):

- `figures/anim_fhn_sweep.mp4` — FHN: $v(t)$ fit from the minimiser at every stage, GP vs control (seed 2), with the running cost and error.
- `figures/anim_fhn_phase.mp4` — FHN: the same sweep in the (v, w) phase plane with the coefficient bars against the truth.
- `figures/anim_fhn_multiple_shooting.mp4` — concept: single shooting drawn forward in time at a wrong p , then the $\kappa = 10$ windows appearing one by one, then node removal ($\kappa = 1 \rightarrow 100$) with the cost of the wrong and true p .
- `figures/anim_fhn_landscape.mp4` — FHN: the 2-D cost landscape in the (v, v^3) plane morphing through every stage of the FULL schedule, with the GP and control paths (seed 2) overlaid (data: `anim_landscape_v_v3.csv`, script `analysis/10_animation_data.jl`).
- `figures/anim_lv_sweep.mp4`, `figures/anim_lv_phase.mp4` — Lotka–Volterra: $x(t), y(t)$ fits, phase plane and coefficients along the sweep, GP vs control (seed 1).

A slide deck telling the story in 8 slides is in `slides/multishooting_story.pptx` (built by `slides/make_deck.py` from these figures and `tables/table_numbers.json`). Every figure exists as PDF and PNG under `figures/`.

Contents

Table of figures	1
1 Introduction	4

2	Definitions (read before the figures)	5
2.1	Model, library and data	5
2.2	Cost function	5
2.3	Arms	5
2.4	Metrics	6
2.5	Symbols and frozen constants	6
3	Provenance and what was executed	7
4	Part A — Concept figures	7
5	Part B — Numerical checks of the theory	14
6	Part C — Cost landscapes	15
7	Part D — Guess propagation: the headline experiment	17
8	Part E — Variations	21
9	Part F — Integrators and solvers inside the loss	27
10	Part G — Defects found and fixed	28
11	Part H — Other systems and closing the loop with the theory	31
12	Gates	33
13	Limitations and honest notes	34
	Reproduction	34

1 Introduction

The code-review report R001 (10 September 2026) established that the repository’s central claim reproduces and listed seven defects. This report is the figure companion for a presentation. It adds three things R001 did not have: proper concept figures for the method (Part A), a broad batch of *fresh* simulations run in parallel (522 optimisation sweeps over 8 seeds and 9 variation axes, plus two other dynamical systems) that support or qualify the theory’s claims (Parts B–F, H), and the quantification *and fix* of the defects, with before/after figures (Part G). Nothing is re-used from R001: every quantity that overlaps (the headline sweep, the F1/F2 numbers) is recomputed here from a frozen snapshot of the sources, and the bug figures execute the pre-fix and post-fix code side by side.

Two things the reader should keep in mind. First, the optimiser seed matters enormously and is reported everywhere: one of the eight random starts lands on the blow-up plateau at the very first window size and never moves under the repository’s flat penalty (§7); it is kept in every statistic rather than dropped. Second, the sweep in the committed driver (`fhn_model_selection_mod 8.jl`) had guess propagation *disabled* and used two different monomial orderings for the sweep and the polish (§10); the driver is fixed in the same commit as this report.

2 Definitions (read before the figures)

Everything used in a caption is defined here, including terms inherited from the manuscript and from R001.

2.1 Model, library and data

The target is FitzHugh–Nagumo,

$$\dot{v} = v - \frac{1}{3}v^3 - w + 0.5, \quad \dot{w} = \frac{1}{12.5}(v + 0.7 - 0.8w), \quad (1)$$

written as a general two-dimensional cubic polynomial system so that model *selection* (which monomials are present) is part of the task:

$$\dot{x}_i = \sum_{j=1}^J p_{ij} \Theta_j(\mathbf{x}), \quad i = 1, 2, \quad \Theta = \{1, w, w^2, w^3, v, vw, vw^2, v^2, v^2w, v^3\}, \quad J = 10. \quad (2)$$

Read this: the library order above is the *multi-index* order produced by `helper_functions.jl` and used by `odefun_poly!` (mod 8). The older hard-coded RHS `odefun` (mods 2–7) uses the order $1, v, w, v^2, vw, w^2, v^3, v^2w, vw^2, w^3$. The two orders are different permutations of the same 10 monomials; mixing them is finding N1 in §10. All coefficient plots here use the library order, with the $\dot{v}$ block (10 entries) followed by the $\dot{w}$ block. The parameter vector $p \in \mathbb{R}^{20}$ has 6 non-zero entries: $\dot{v}$: $(1)=0.5$, $(w)=-1$, $(v)=1$, $(v^3)=-1/3$; $\dot{w}$: $(1)=0.056$, $(w)=-0.064$, $(v)=0.08$ (seven including the $\dot{w}$ constant). The optimisation vector is $z = [x_0; p] \in \mathbb{R}^{22}$: the first two entries are a free initial condition that enters only through the term $\|x_0 - y_0\|^2$.

Data follow the repository recipe exactly (frozen mod 8 lines 103–157, reproduced to 6×10^{-15} , gate G3): integrate with the in-house fixed-step Tsit5 at $\delta t_{\text{fine}} = 0.01$ from $(1, 1)$ to $T = 156$, keep every 100th point, crop to $t \in [50, 150]$ and shift to $t \in [0, 100]$ ($N = 101$ samples, spacing $\Delta t = 1$), and add Gaussian noise to each component with standard deviation $0.05 \times$ that component’s own standard deviation (seed 1287436679). The resulting maximum noise norm is $\|\eta\|_{\max} = 0.159$ and the noise floor of the data term is $\frac{1}{N} \sum_i \|\eta_i\|^2 = 0.0052$.

2.2 Cost function

For a window size κ (number of data intervals per window, $1 \leq \kappa \leq N-1$) the record is cut into $K = \lceil (N-1)/\kappa \rceil$ windows; each window is integrated *from the noisy datum at its left node* with S fixed Tsit5 sub-steps of length $\delta t = \Delta t/S$ per data interval, and

$$J_\kappa(z) = \frac{1}{N} \left[\|x_0 - y_0\|^2 + \sum_{i=1}^{N-1} \|\hat{x}(t_{i+1}; p, y_{\tau(i)}) - y_{i+1}\|^2 \right] + \frac{\gamma}{20} \sum_{j=1}^{20} \text{smooth } \ell_1(p_j), \quad (3)$$

where $\tau(i)$ is the last node at or before t_i , and $\text{smooth } \ell_1(x) = \alpha^{-1} [\log(1 + e^{-\alpha x}) + \log(1 + e^{\alpha x})]$ with $\alpha = 500$ (equal to $|x|$ for $|\alpha x| > 40$). $\kappa = 1$ is one window per data interval; $\kappa = N - 1 = 100$ is single shooting. **Read this:** node states are *not* optimisation variables and no continuity constraint is imposed (unlike textbook multiple shooting); this is the manuscript’s formulation and the repository’s code. If any state exceeds 10^3 in magnitude or becomes NaN during integration, the loss returns the constant 10^3 (*flat blow-up penalty*, a plateau with zero gradient). The *graded* penalty (a variant introduced here) returns $10^3 (1 + \frac{N-1-i}{N-1}) + J_{\text{partial}}/N$ when the blow-up occurs at data index i , so that later blow-ups cost less and a derivative-free optimiser has a direction. The manuscript’s J_K (no normalisation, no sparsity term) differs from (3) by the factor $1/N$ and the γ term; the theory figures (Part B) use $\gamma = 0$.

2.3 Arms

GP (guess propagation) Sweep κ upward through a schedule; the minimiser of each stage is the start of the next. Schedules: FULL = $\{1, 2, 3, 4, 5, 6, 8, 10, 12, 15, 20, 25, 33, 50, 75, 100\}$ (default); SHORT = $\{1, 2, 5, 10, 25, 50, 100\}$ for the variation experiments; DENSE = $\{1, \dots, 30, 35, 40, \dots, 100\}$ (44 stages); COARSE = $\{1, 5, 25, 100\}$; JUMP = $\{1, 100\}$.

best-GP As GP but the start of the next stage is the best (lowest-cost) minimiser seen so far.

control (no propagation) Every stage starts from the same random seed vector. This is the arm the committed mod 8 driver was running.

optimiser Nelder–Mead (Optim.jl, 2500 iterations, default tolerances) unless stated; BFGS and L-BFGS use the exact ForwardDiff gradient of (3) through the integrator (possible only after fix F4), same iteration cap, $g_{\text{tol}} = 10^{-8}$.

seed Optimiser seed $s \in \{1, \dots, 8\}$: $z_{\text{seed}} = [y_0; 0.1\xi]$, $\xi \sim \mathcal{N}(0, I_{20})$ from `MersenneTwister(s)`; $\|p_0 - p^*\| \in [1.50, 1.84]$ over the 8 seeds (0.1 is R001’s scale; the repository uses 0.01 — see 28).

Read this: arms are compared at *equal* budgets (same iterations, same schedule, same seeds) except where the figure says otherwise: the schedule experiment (21) deliberately changes the number of stages, and the optimiser experiment (25) reports wall time because the gradient methods do different work per iteration.

2.4 Metrics

J_κ (**primary**) the cost (3) at the returned minimiser, including the sparsity term; $J_\kappa \geq 10^3$ flags a blow-up (*plateau*). Costs at different κ are not directly comparable (different restarts).

$\|p - p^*\|$ Euclidean distance between the fitted 20-vector and the true coefficients; the identification metric. Not computed by the repository; the same definition as R001.

recovery score fractional system-recovery score in $[0, 1]$: each of the 20 coefficients earns credit 1 if it is correctly zero ($|p_j| < 0.05$ when $p_j^* = 0$) or correctly non-zero within 25 % of the true value; the score is the mean credit. *support recall* (fraction of the 7 true non-zeros found) and *false positives* (true zeros with $|p_j| \geq 0.05$) are its components.

paired difference for two arms sharing a seed, the per-seed difference of $\|p - p^*\|$ at a given κ ; reported as the mean with a 95 % percentile bootstrap CI over 2000 resamples of the *seeds* (resampling unit = optimiser seed). “Cellwise” counts report in how many seeds one arm is strictly better.

wall time seconds on one core for one stage; only used comparatively and never in a gate.

2.5 Symbols and frozen constants

symbol	meaning	value	where frozen
N	number of samples	101	data recipe (mod 8)
Δt	sample spacing	1.0	data recipe
$S, \delta t$	integrator sub-steps per interval, step	10, 0.1	driver (27 varies S)
κ, K	window size (data intervals), number of windows	swept, $\lceil 100/\kappa \rceil$	schedules above
γ, α	sparsity weight, smooth- ℓ_1 sharpness	5×10^{-2} , 500	driver (24 varies γ)
σ_{rel}	relative noise per component	0.05	data recipe (23 varies)
$\ \eta\ _{\text{max}}$	largest noise norm in the data	0.159	<code>fhn_data_meta.json</code>
L	Lipschitz constant $\sup \ \partial f / \partial x\ _2$ along the true orbit / on the box $[-2.5, 2.5] \times [-1, 2]$	3.05 / 5.34	<code>concept_meta</code>
μ	logarithmic norm $\sup \lambda_{\text{max}}((J + J^\top)/2)$ along the orbit	1.17	idem
$\tilde{L}$	$\sup \ \partial f / \partial p\ _2$ along the orbit (Lemma 2)	10.56	idem
$\Delta T, \Delta T_1, \Delta T_2$	window length $\kappa \Delta t$; fine window length; $\tau_k - \tau_k^-$ (Prop. 1)	—	manuscript
blow-up threshold	$ x > 10^3$ or NaN	10^3	loss

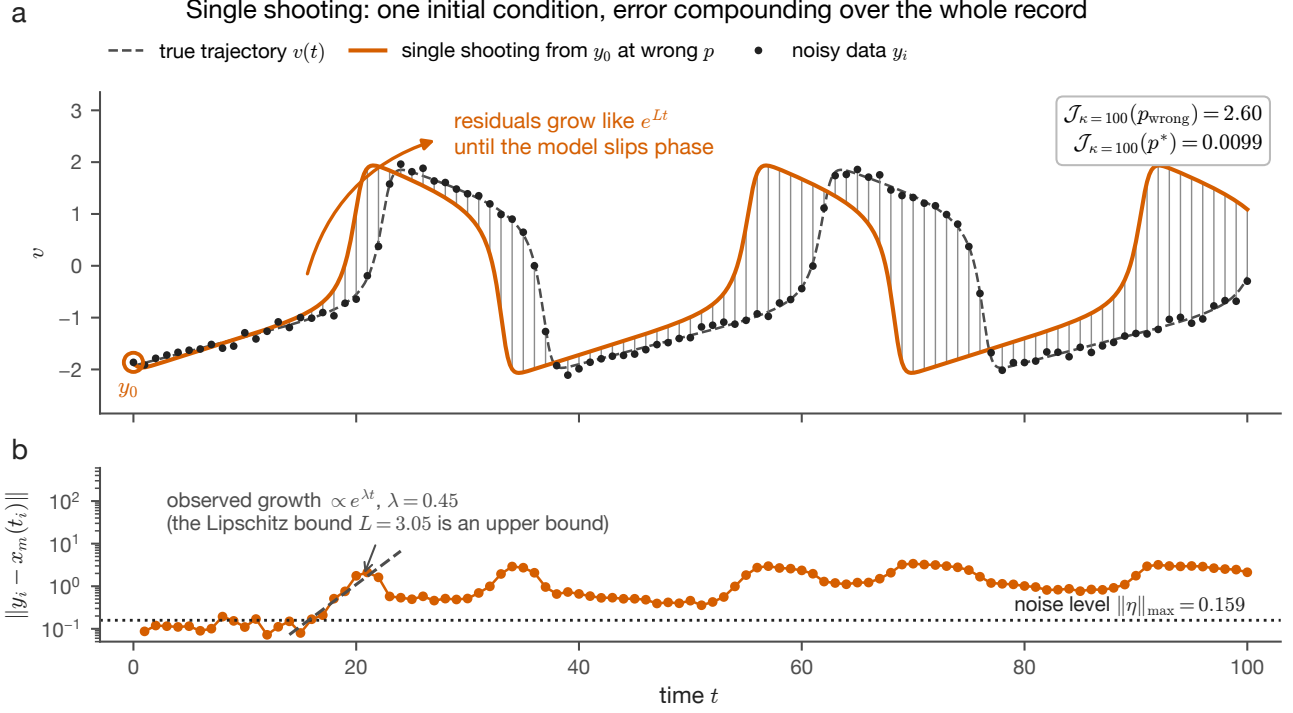
Read this: L is measured, not assumed. Along the FHN limit cycle the actual amplification of an initial-state perturbation never exceeds ≈ 8 (8), so every bound that carries $e^{L\Delta T}$ is loose by orders of magnitude for $\Delta T \gtrsim 2$; the bounds are qualitatively right (they order the arms correctly) but not quantitatively tight.

3 Provenance and what was executed

All numerics run from the frozen snapshot `deps/` (post-fix sources) through `analysis/common.jl` and `analysis/hs_core.jl`, a corrected, allocation-light, dimension-generic re-implementation of the windowed loss that is gated against the frozen mod 8 `forward_simulation_loss_windows` (relative difference 1.9×10^{-13} over 42 evaluations, gate G2) and against the frozen mod 8 data block (6×10^{-15} , G3). The bug scripts additionally execute `deps/prefix/` (pre-fix sources, byte-identical to commit 44abf4a, `origin/main` at freezing time — `git fetch` was run first). SHA-256 of both snapshots: `deps/MANIFEST.md` (G1). Julia 1.11.3 with the committed environment and no package added; figures by matplotlib from CSV/JSON; nothing in Julia plots. `Distributed` cannot open sockets in the sandbox, so the 522 sweep jobs ran as 12+8+6 independent sharded processes (`REPRODUCE.md`). Realised coverage (every axis value and cell count, computed by `analysis/make_inventory.py`) is in `planning/coverage_matrix.md`; result-file hashes in `analysis/results/results_hashes.json`; every number quoted in prose is listed with its source in `analysis/number_manifest.md`.

4 Part A — Concept figures

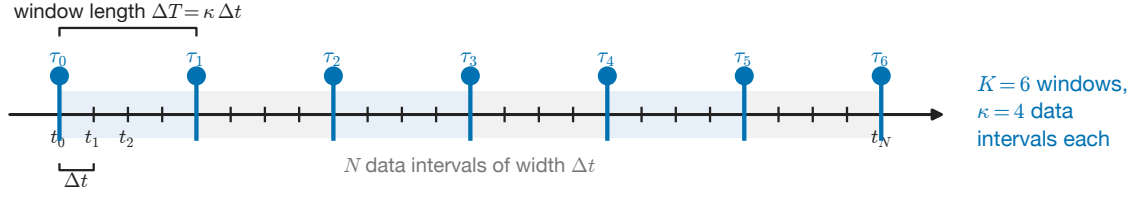
These are the slide figures. They use the real FHN data of §2 and a deliberately *wrong* parameter vector p_{wrong} (cubic coefficient -0.30 instead of $-1/3$, $\hat{w}$ time-scale multiplied by 1.15; $\|p_{\text{wrong}} - p^*\| = 0.038$) so that the single-shooting trajectory visibly drifts while the windowed segments stay close.



p_{wrong} : the true p^* with the v^3 coefficient $-1/3 \rightarrow -0.3$ and the w block scaled by 1.15 ($\|p - p^*\| = 0.0377$); noise floor $\mathcal{J} = 0.0052$

Figure 2: Single shooting. The model is integrated once from the first datum y_0 over the whole record with the wrong parameter p_{wrong} (vermilion); thin vertical lines are the residuals to the data. The residual (right, log scale) sits at the noise level until the model slips phase near $t \approx 18$, then grows at an observed rate ≈ 0.45 per time unit (the Lipschitz bound $L = 3.05$ is an upper bound, far above it) and saturates at the size of the attractor; the cost is $\mathcal{J}_{100}(p_{\text{wrong}}) = 2.60$ (vs 0.0099 at p^*), so a tiny parameter error produces a large, non-convex cost. Data: `concept_segments.csv` (`param=wrong`, `window_size=100`), `concept_costs.csv`.

a Fine partition: a shooting node τ_k at every κ -th datum; each window restarts from the datum $y_{\tau_{k-1}}$



b Node removal: delete every other node $\Rightarrow$ half as many windows, each twice as long

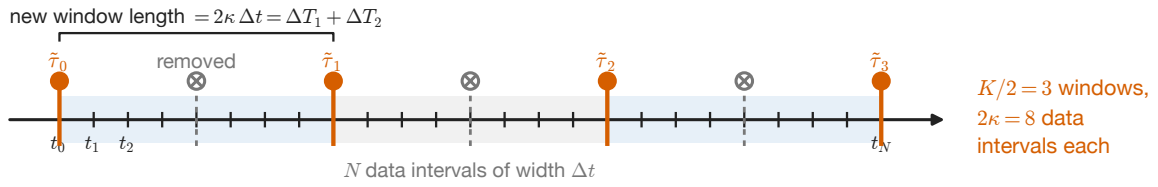


Figure 3: The shooting partition. Top: the data times $t_0, \dots, t_N$ (ticks, spacing Δt) and the shooting nodes $\tau_0 < \tau_1 < \dots < \tau_K$ placed at every κ -th datum; the windows $[\tau_{k-1}, \tau_k]$ of length $\Delta T = \kappa \Delta t$ are shaded alternately. Bottom: the coarser partition obtained by removing every other node ($K/2$ windows of length $2\Delta T$) — the elementary step of guess propagation. Schematic (no data); replaces the manuscript's `shooting_partition.png`.

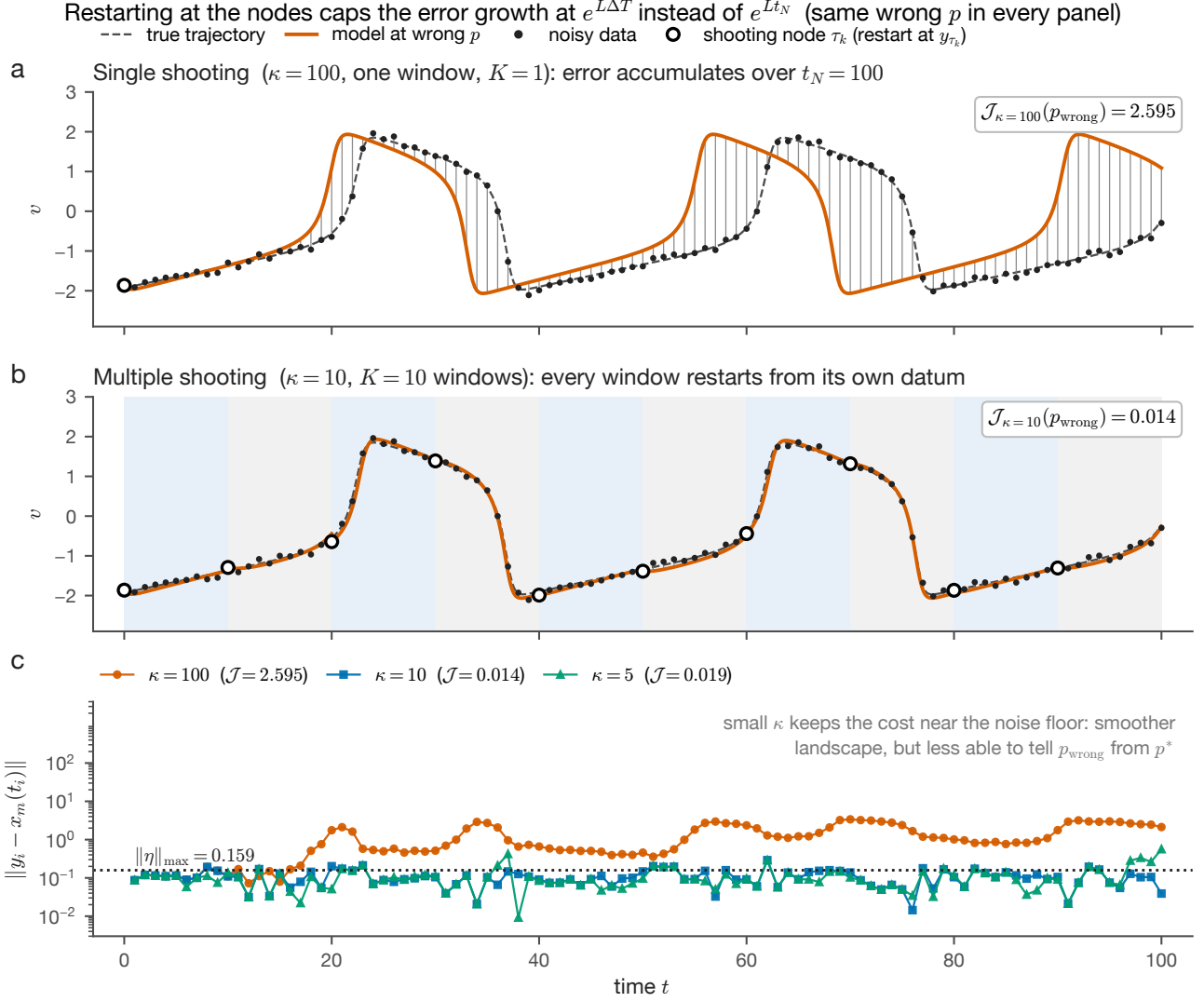
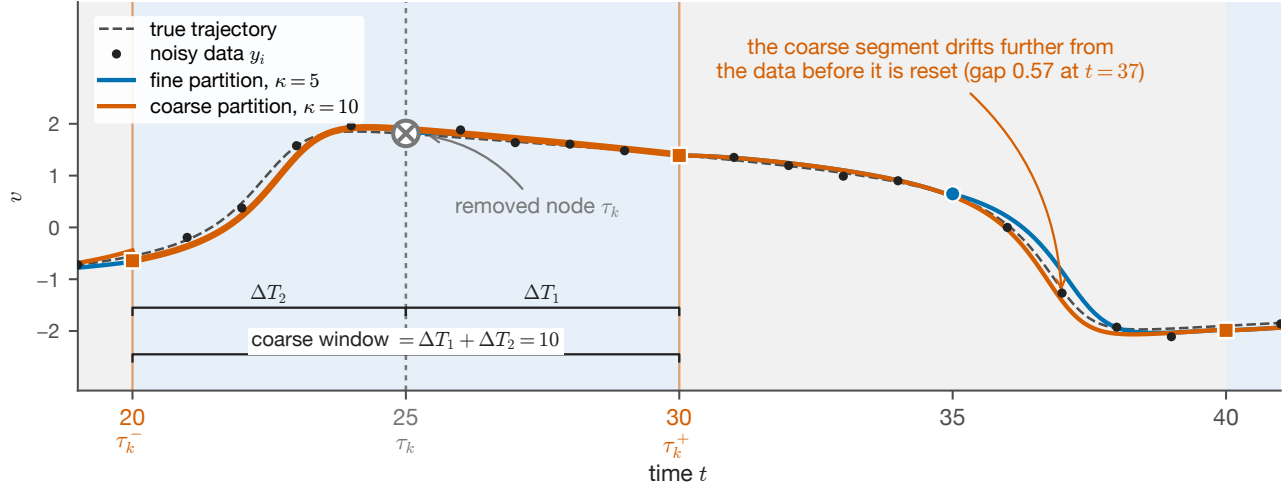


Figure 4: Multiple shooting. The same wrong parameter as 2, integrated (a) as single shooting ($\kappa = 100$, $J = 2.60$) and (b) restarting from the datum at every 10th sample ($\kappa = 10$, $J = 0.0140$); windows are shaded, node markers highlighted, residual lines drawn to the data; (c) the per-datum residual magnitude on a log scale for $\kappa = 100, 10, 5$ ($J = 2.60, 0.0140, 0.0188$). Restarting resets the accumulated error at every node, so the exponential amplification is paid over $\Delta T = \kappa \Delta t$ instead of over the whole record; with $\kappa \leq 10$ the wrong model is only 1.4–1.8 $\times$ the cost of the true one, with $\kappa = 100$ it is 260 $\times$. The flip side, visible in (c): short windows keep even a wrong model near the noise floor, i.e. they give a smooth landscape at the price of less power to discriminate p from p^* — which is why the sweep must end at single shooting. Replaces `multiple_shooting.png`. Data: `concept_segments.csv`, `concept_costs.csv`.

Removing the node τ_k : the segment launched from τ_k^- must now cover $[\tau_k^-, \tau_k^+]$



$$\|\hat{\mathcal{J}}_K(p) - \mathcal{J}_K(p)\| \leq 2 |I_R| C_{\max} \left(e^{L(\Delta T_1 + \Delta T_2)} \|\eta\|_{\max} + e^{L\Delta T_1} \left[\frac{\tilde{L}}{L} (e^{L\Delta T_2} - 1) \|p - p^*\| + \|\eta\|_{\max} \right] \right)$$

$\Delta T_1 = \max_k(\tau_k - \tau_{k-1})$ = fine window length; $\Delta T_2 = \max_k(\tau_k - \tau_k^-)$ = distance from the removed node to its left neighbour (both = 5 here); $|I_R|$ = number of removed nodes; $C_{\max}$ = largest segment/data magnitude over the affected windows $L = 3.05$, $\tilde{L} = 10.56$ (Lipschitz in x , in p); $\|\eta\|_{\max} = 0.159$. Background shading: the coarse ($\kappa = 10$) windows.

Figure 5: Node removal (Proposition 1). Zoom on $t \in [19, 41]$ with the wrong parameter: the fine partition ($\kappa = 5$, blue segments) and the coarse one after removing the node τ_k at $t = 25$ (open marker; $\kappa = 10$, vermilion), with $\tau_k^- = 20$ and $\tau_k^+ = 30$; the measured gap between the fine and coarse segments at $t = 37$ is printed. After removal the segment launched at the left neighbour τ_k^- must cover $[\tau_k^-, \tau_k^+]$; the cost changes only on the windows adjacent to the removed nodes, by at most $2|I_R| C_{\max} \left(e^{L(\Delta T_1 + \Delta T_2)} \|\eta\|_{\max} + e^{L\Delta T_1} \left[\frac{\tilde{L}}{L} (e^{L\Delta T_2} - 1) \|p - p^*\| + \|\eta\|_{\max} \right] \right)$ with ΔT_1 the fine window length and $\Delta T_2 = \tau_k - \tau_k^-$. Design rule: remove few nodes at a time, prefer low-noise nodes, keep ΔT_2 small. Replaces `comparison_shooting_nodes.png`.

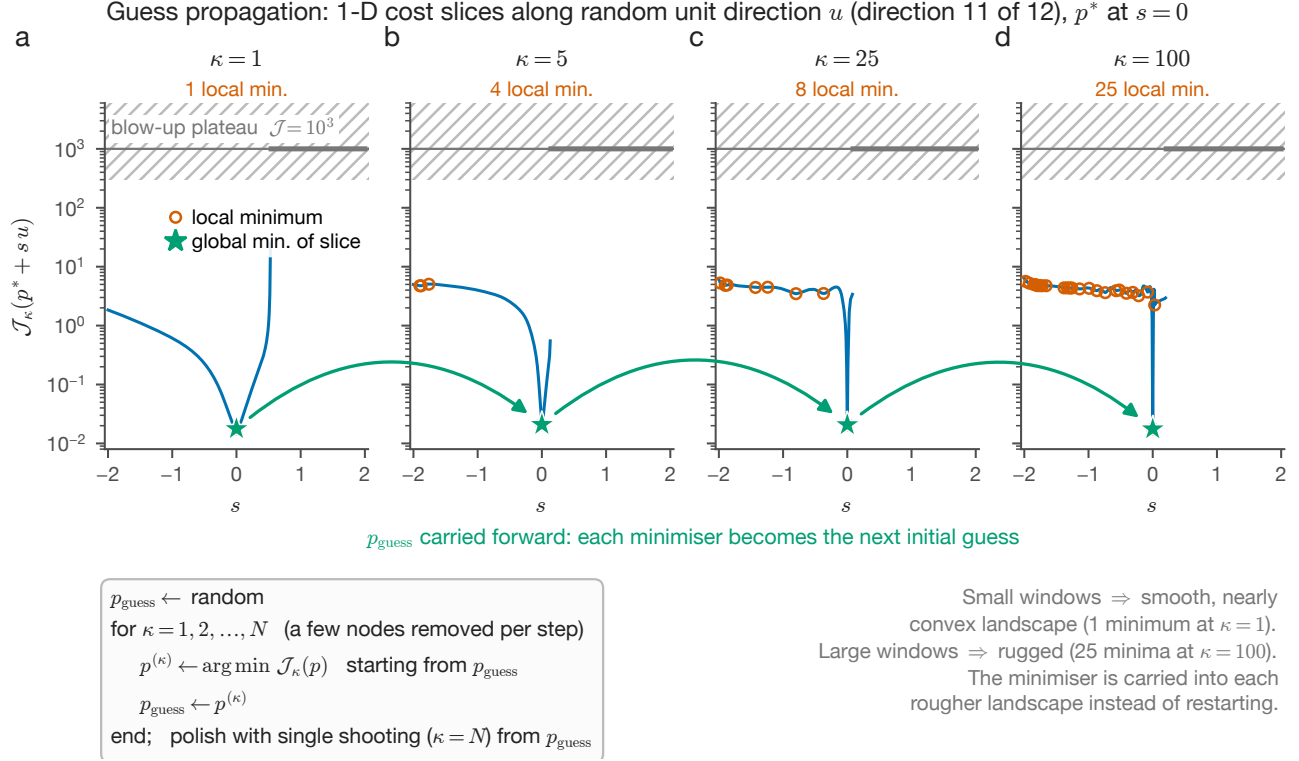


Figure 6: The guess-propagation algorithm. One-dimensional slices $J_\kappa(p^* + s u)$ along the same random unit direction u (the one of the 12 directions of 13 with the most local minima at $\kappa = 100$: 1, 4, 8, 25 local minima at $\kappa = 1, 5, 25, 100$, circled) for $\kappa = 1, 5, 25, 100$ (log scale; hatched band = blow-up plateau $J = 10^3$; star = global minimum, which is at $s = 0$ because the slices pass through p^*). Small windows give a smooth, nearly convex landscape; large windows add local minima and a wide plateau. The algorithm solves the $\kappa = 1$ problem from a random start and carries each minimiser (arrow) into the next, more rugged landscape, ending with a single-shooting polish. Data: `landscape_1d_slices.csv`.

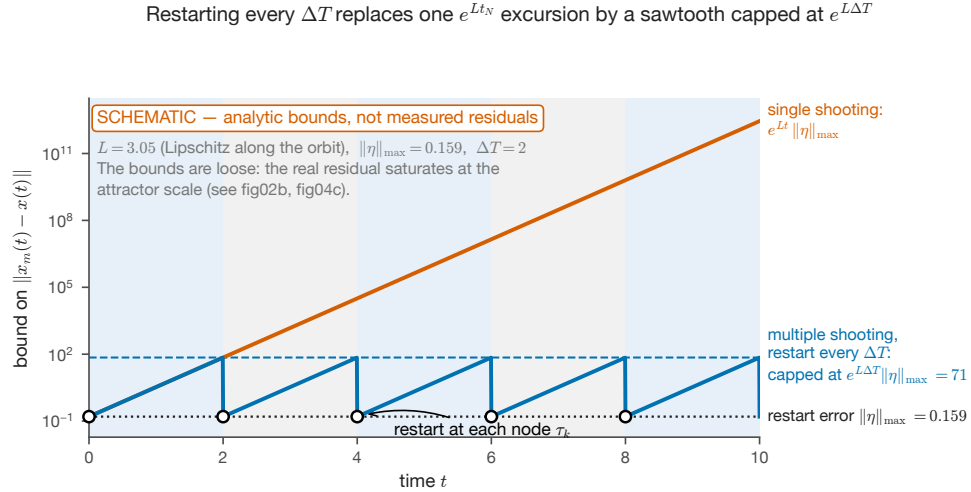


Figure 7: Error growth, schematic. The Lemma 1 envelope $e^{L t} \|\eta\|_{\max}$ for single shooting (one restart at t_0) versus the sawtooth of multiple shooting, capped at $e^{L \Delta T} \|\eta\|_{\max}$ by the restart at every node, drawn with the report's $L = 3.05$ and $\|\eta\|_{\max} = 0.159$. This is the manuscript's one-line argument; 8 shows how loose the envelope is on the actual system.

5 Part B — Numerical checks of the theory

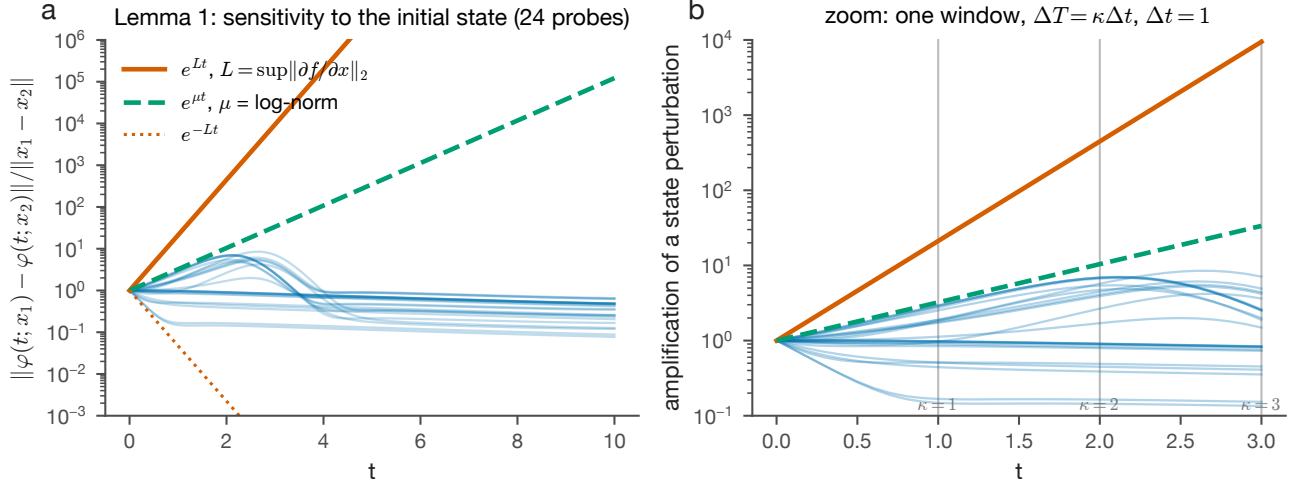


Figure 8: Lemma 1 (flow sensitivity to the initial state). Amplification $\|\varphi(t; p^*, x_1) - \varphi(t; p^*, x_2)\| / \|x_1 - x_2\|$ for 24 probes (x_1 = a datum at 4 different phases of the orbit, $x_2 = x_1 + 10^{-3}u$ for 6 random unit directions; blue), against e^{Lt} with $L = \sup_{\text{orbit}} \|\partial f / \partial x\|_2 = 3.05$ (vermilion), e^{-Lt} (dotted) and $e^{\mu t}$ with the logarithmic norm $\mu = 1.17$ (green dashed). (a) 10 time units, (b) zoom on the first three windows. The two-sided bound holds everywhere, but the actual amplification peaks at ≈ 8 near the fast jump of the relaxation oscillation and then *contracts* (the limit cycle is attracting), whereas e^{Lt} reaches 10^{13} at $t = 10$. Data: lemma1_flow_sensitivity.csv, concept_meta.json.

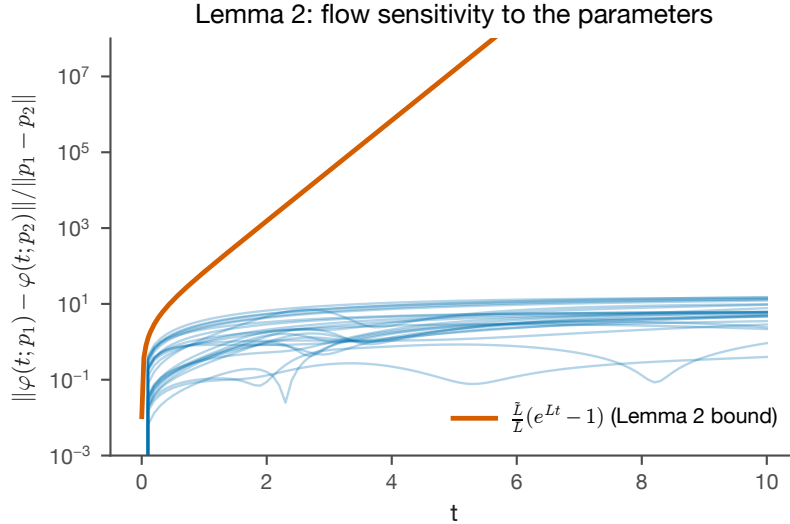


Figure 9: Lemma 2 (flow sensitivity to the parameters). $\|\varphi(t; p_1, x_0) - \varphi(t; p_2, x_0)\| / \|p_1 - p_2\|$ for 24 probes ($p_2 = p^* + 10^{-3}u$, random unit $u \in \mathbb{R}^{20}$) versus the bound $\frac{\tilde{L}}{L}(e^{Lt} - 1)$ with $\tilde{L} = \sup_{\text{orbit}} \|\partial f / \partial p\|_2 = 10.56$. The bound holds; the measured sensitivity saturates around 10^1 – 10^2 because the orbit is attracting. Data: lemma2_param_sensitivity.csv.

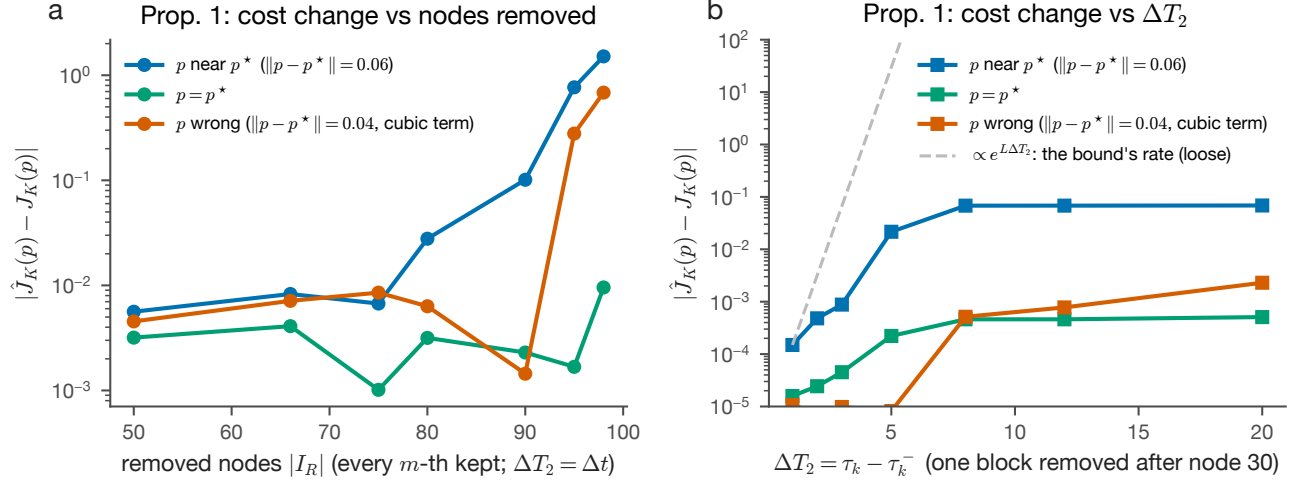


Figure 10: Proposition 1 (node removal changes the cost by a bounded amount). Starting from the finest partition (every datum a node, $K = 100$) with $\gamma = 0$: (a) keep every m -th node so that $|I_R| = 100 - \lceil 100/m \rceil$ nodes are removed ($\Delta T_2 = \Delta t$ fixed, $\Delta T_1 = m\Delta t$ grows), and (b) remove one contiguous block of b nodes after node 30 ($|I_R| = b$, $\Delta T_2 = b\Delta t$ grows). Shown for $p = p^*$ (green), a vector 0.02 away from p^* on 6 coefficients (blue, $\|p - p^*\| = 0.06$) and p_{wrong} of Part A (vermillion, $\|p - p^*\| = 0.04$ concentrated on the cubic term). $|\hat{J} - J|$ grows with both $|I_R|$ and ΔT_2 , and much faster for the wrong parameter, as the proposition says; the bound's rate $e^{L\Delta T_2}$ (grey dashed in (b)) is far steeper than the observed growth, which saturates once the removed block spans a full fast excursion. Data: `prop1_node_removal.csv`.

6 Part C — Cost landscapes

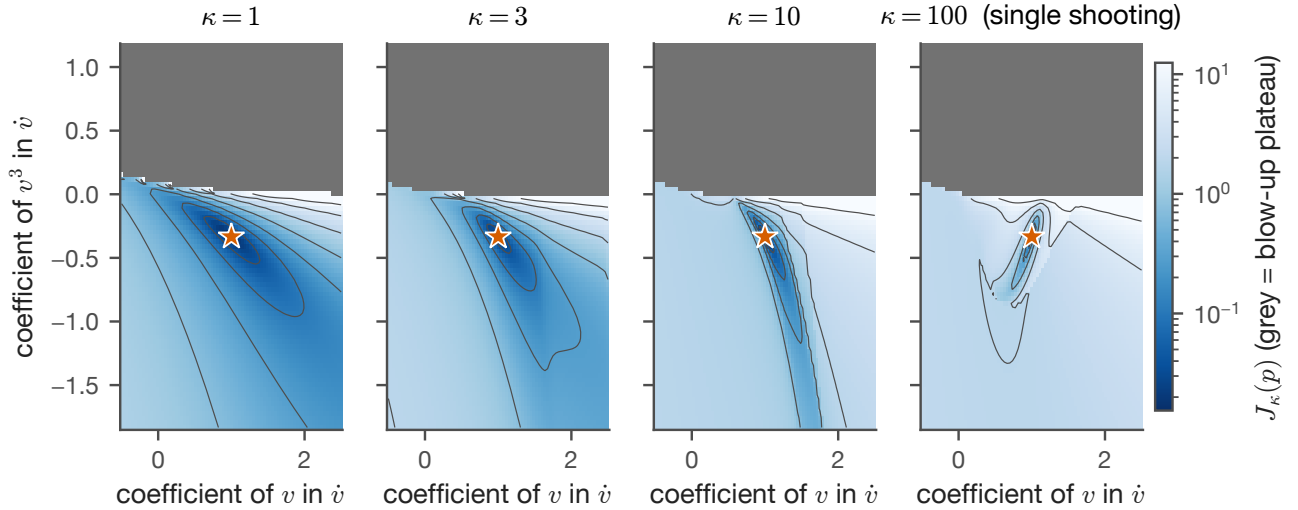


Figure 11: 2-D slices of J_κ in the (v, v^3) coefficient plane of the $\dot{v}$ equation. All other coefficients at their true values, $x_0 = y_0$, $\gamma = 0.05$; star = truth $(1, -1/3)$; grey = blow-up plateau ($J \geq 10^3$); colour is $\log J$. As κ grows from 1 to 100 the basin around the truth narrows by an order of magnitude and the plateau expands: any positive cubic coefficient (above the horizontal edge) makes $\dot{v} \sim cv^3$ blow up in finite time, and at $\kappa = 100$ even v -coefficients > 1.6 do. This is the geometric reason the control arm fails at large κ . Data: `landscape_2d_v_v3.csv`.

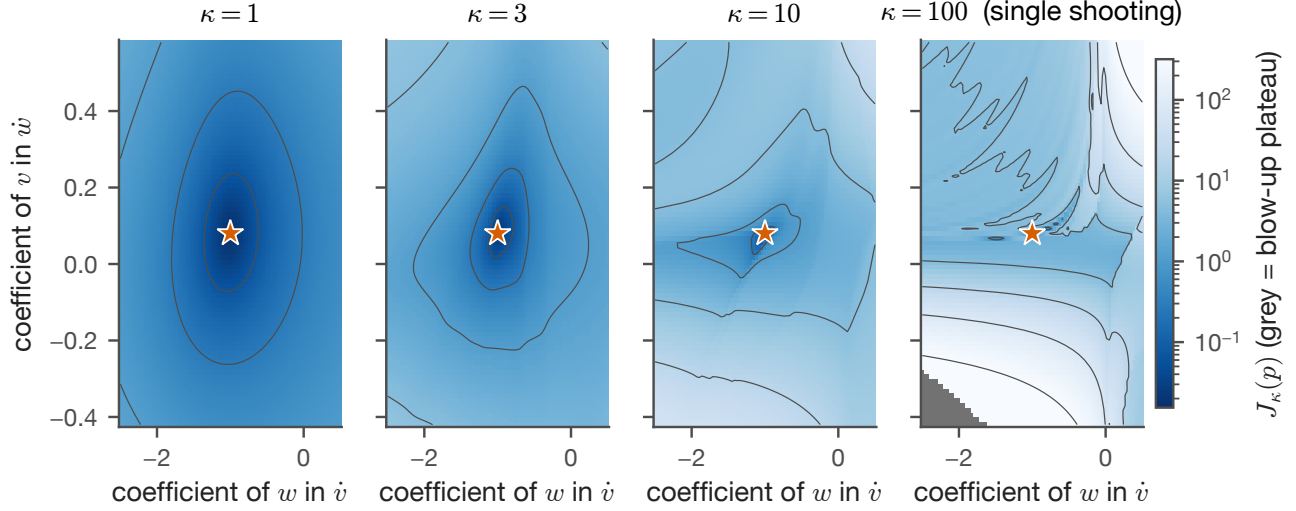


Figure 12: 2-D slices in the $(w \text{ in } \dot{v}, v \text{ in } \dot{w})$ plane. Same protocol as 11; star = truth $(-1, 0.08)$. The $\dot{w}$ coefficient (vertical axis, true value 0.08) is a slow time-scale: at $\kappa = 1$ the cost is almost flat along it (weakly identifiable from one-interval windows), and only long windows ($\kappa \geq 10$) pin it down — the mechanism behind the non-monotone parameter error along the sweep (15b). Data: `landscape_2d_w_wv.csv`.

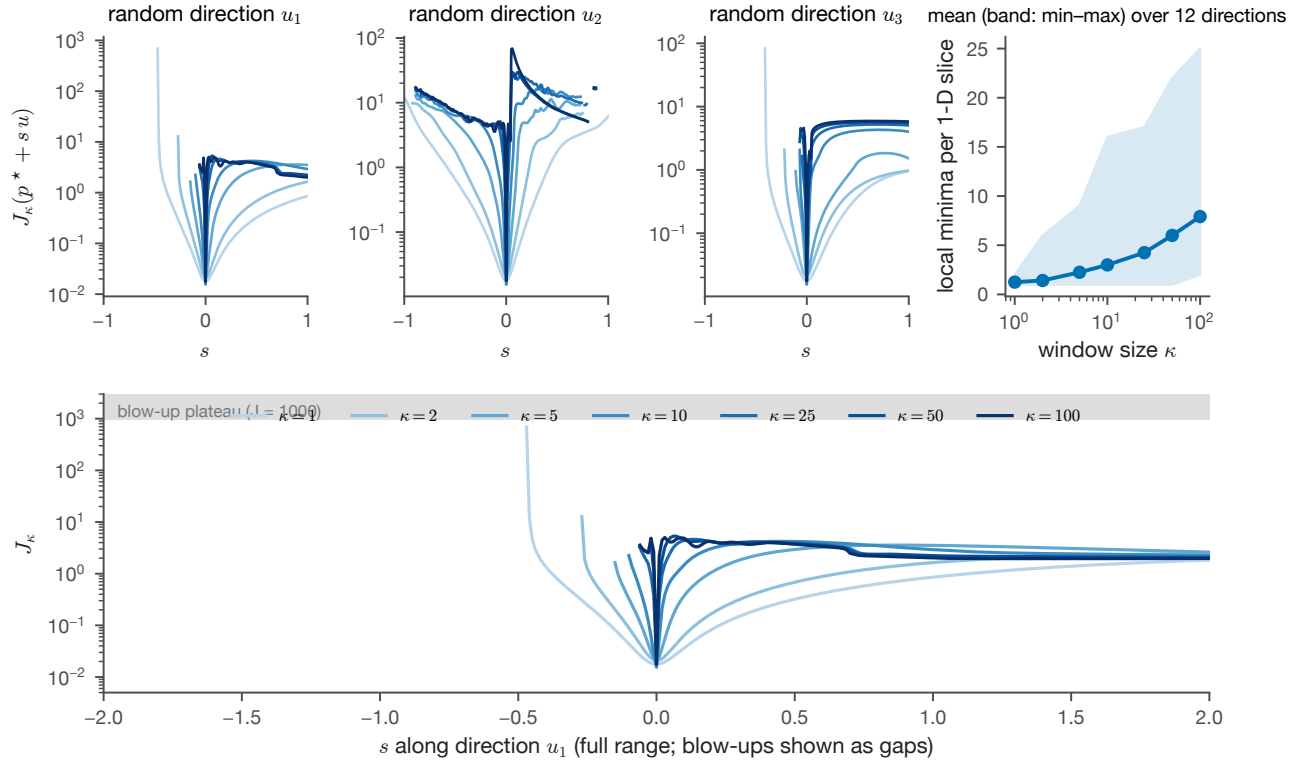


Figure 13: 1-D slices along random directions and the number of local minima. Top row: $J_\kappa(p^* + s u_d)$ for three random unit directions $u_d \in \mathbb{R}^{20}$ (colour light to dark = $\kappa = 1, 2, 5, 10, 25, 50, 100$; gaps = plateau); top right: local minima per slice (mean over 12 directions, band = min-max) rising from 1.25 at $\kappa = 1$ to 7.9 at $\kappa = 100$. Bottom: the full $s \in [-2, 2]$ range for direction 1. Larger windows sharpen the minimum (better conditioning at the truth) but add spurious minima and cut the finite region down to $|s| \lesssim 0.5$. Data: `landscape_1d_slices.csv`, `landscape_1d_minima.csv`.

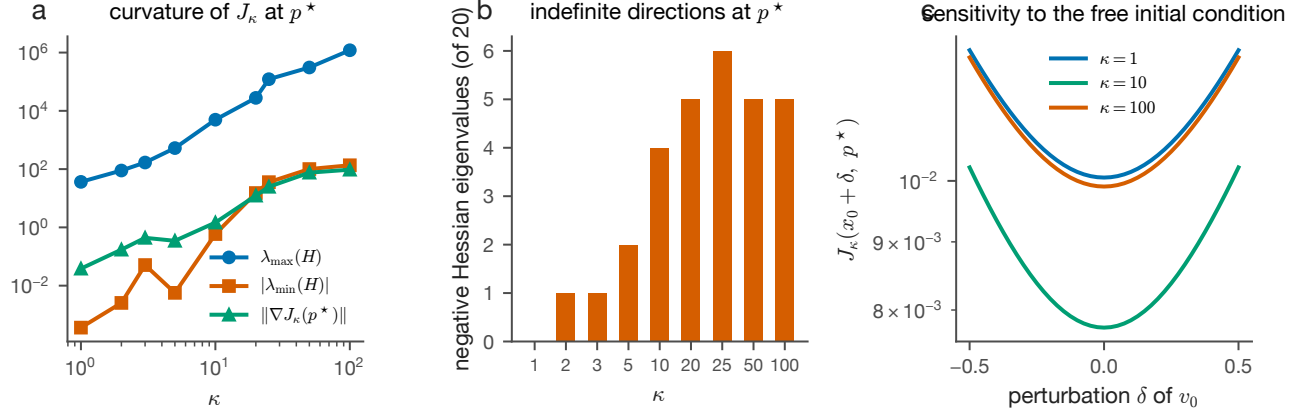


Figure 14: Curvature of the data term at p^* versus window size. (a) Largest and smallest Hessian eigenvalues of the data term of J_κ at p^* (ForwardDiff, 20 parameters) and the gradient norm: $\lambda_{\max}$ grows from 36 ($\kappa = 1$) to 1.2×10^6 ($\kappa = 100$) while $\|\nabla J_\kappa(p^*)\|$ grows from 0.04 to 96 — with noisy data the truth is not a stationary point and drifts away from the minimiser as κ grows. (b) The number of negative eigenvalues: the noisy cost is indefinite at p^* for every $\kappa \geq 2$ (up to 6 of 20 directions), so the manuscript’s strong-convexity assumption can only hold in a neighbourhood of the *minimiser* $p^{(\kappa)}$, not of p^* (checked there in 37). (c) Sensitivity of J_κ to the free initial-condition variable v_0 : a shallow parabola for every κ , because x_0 only enters through the first residual. Data: `hessian_at_ptrue.csv`, `landscape_x0_sensitivity.csv`.

7 Part D — Guess propagation: the headline experiment

Protocol: 8 optimiser seeds $\times$ 3 arms $\times$ the 16-stage FULL schedule, Nelder–Mead with 2500 iterations per stage, $\gamma = 0.05$, $S = 10$, flat penalty. Table 2 gives the seed medians and the paired statistics per stage.

Table 2: Headline sweep, seed medians over 8 seeds (GP = guess propagation, best-GP = best-guess propagation, control = no propagation) and the paired difference GP – control in $\|p - p^*\|$ with its 95 % bootstrap CI; “#GP better” counts seeds where GP is strictly better; “#GP on plateau” counts GP seeds whose minimiser is a blow-up (at $\kappa = 1$ three seeds are on the plateau initially; two escape at $\kappa = 2$ because the plateau at $\kappa = 1$ is not flat for their simplex — see text). Generated by `tables/make_tables.py` from `sweeps.csv`.

κ	J GP	J best-GP	J control	$\ p - p^*\ $ GP	$\ p - p^*\ $ best-GP	$\ p - p^*\ $ control	score GP	score control	GP–control	CI lo	CI hi	#GP better	#GP on plateau
1	0.02569	0.02569	0.02569	0.8641	0.8641	0.8641	0.4	0.4	0	0	0	0	3
2	0.02081	0.02081	0.03726	0.3803	0.3803	1.314	0.75	0.275	-0.7724	-1.112	-0.4196	7	1
3	0.01864	0.019	0.05955	0.3748	0.3893	1.466	0.75	0.225	-0.8679	-1.309	-0.4865	7	1
4	0.01549	0.01577	0.09718	0.3325	0.3615	1.51	0.775	0.25	-0.9232	-1.248	-0.5652	7	1
5	0.01906	0.01889	0.1259	0.2218	0.221	1.543	0.775	0.2	-1.133	-1.654	-0.6627	7	1
6	0.02114	0.02054	0.2428	0.3441	0.4522	1.716	0.55	0.2	-1.159	-1.526	-0.7401	7	1
8	0.0161	0.0154	0.3185	0.3593	0.3964	1.59	0.675	0.275	-1.113	-1.36	-0.7559	7	1
10	0.01406	0.01438	0.3086	0.2995	0.3347	1.737	0.8	0.2	-1.234	-1.634	-0.7787	7	1
12	0.01556	0.0155	0.6561	0.278	0.3312	1.622	0.8	0.25	-1.163	-1.415	-0.7988	7	1
15	0.01414	0.01409	0.5398	0.3348	0.3402	1.574	0.725	0.275	-1.109	-1.327	-0.7739	7	1
20	0.01436	0.01459	0.5358	0.2425	0.2709	1.612	0.8	0.25	-1.236	-1.547	-0.8214	7	1
25	0.01345	0.01362	0.861	0.3113	0.3455	1.534	0.85	0.25	-1.111	-1.401	-0.7455	7	1
33	0.01375	0.01373	0.8233	0.3372	0.3758	1.498	0.825	0.225	-0.9927	-1.358	-0.5627	7	1
50	0.01327	0.01334	1.334	0.3353	0.3393	1.807	0.8	0.2	-1.63	-2.426	-0.9189	7	1
75	0.0128	0.01289	1.392	0.3223	0.3179	1.793	0.8	0.225	-1.167	-1.522	-0.7642	7	1
100	0.01273	0.01288	1.506	0.3216	0.3105	1.649	0.8	0.25	-1.138	-1.426	-0.7719	7	1

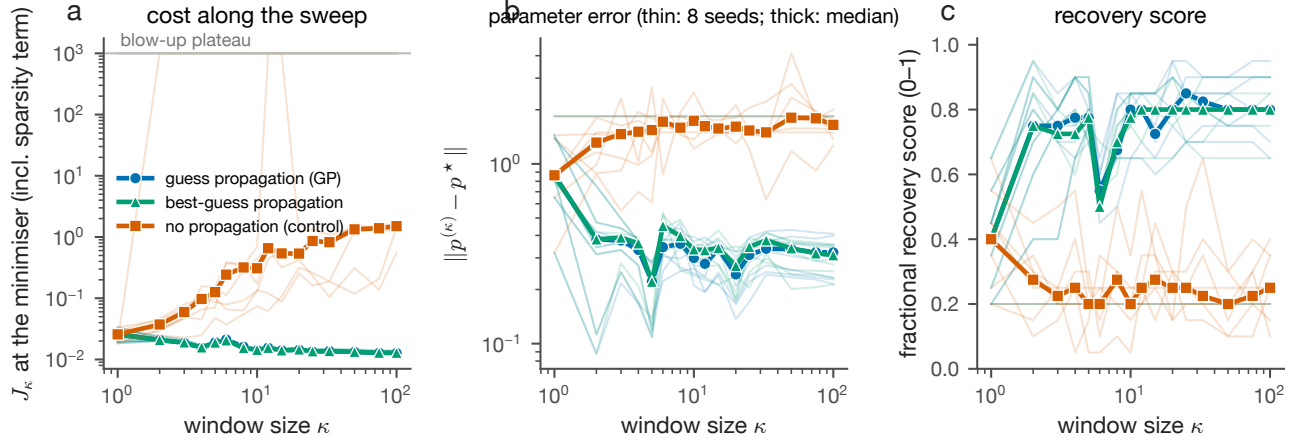


Figure 15: Guess propagation versus no propagation, 8 seeds. (a) Cost, (b) parameter error and (c) recovery score at the minimiser of each stage; thin lines = individual seeds, thick = median; blue circles GP, green triangles best-GP, vermilion squares control. All arms coincide at $\kappa = 1$ (same start). From $\kappa = 2$ on, GP drops to $\|p - p^*\| \approx 0.22\text{--}0.38$ and ends single shooting at 0.32 (score 0.80), whereas the control climbs back to the seed distance (1.65 at $\kappa = 100$; 25 % of its seeds end on the plateau, all others in spurious minima with J up to 1.5). One GP seed (seed 1) starts on the plateau at $\kappa = 1$ and never moves (flat line at 1.84). The parameter error is *not* monotone: it is smallest at $\kappa = 5$ (0.22) and drifts up to 0.32 while the cost keeps falling — the noise-induced bias of 14. Data: `sweeps.csv` (`exp=main`).

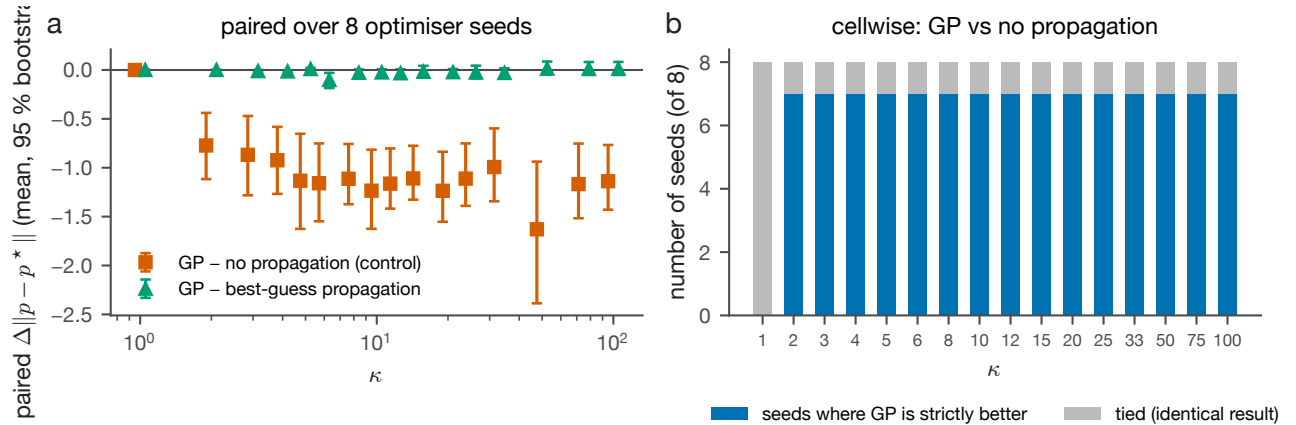


Figure 16: Paired comparison. (a) Mean paired difference in $\|p - p^*\|$ (GP minus control, vermilion squares; GP minus best-GP, green triangles) with 95 % bootstrap CIs over the 8 seeds; the GP-control difference is -0.77 at $\kappa = 2$, -1.14 $[-1.43, -0.77]$ at $\kappa = 100$, and its CI excludes zero at every $\kappa \geq 2$; GP and best-GP are statistically indistinguishable. (b) Cellwise counts: GP is strictly better than the control in 7 of 8 seeds at every $\kappa \geq 2$ and tied in the eighth (the plateau seed). Data: `derived_paired_main.csv`.

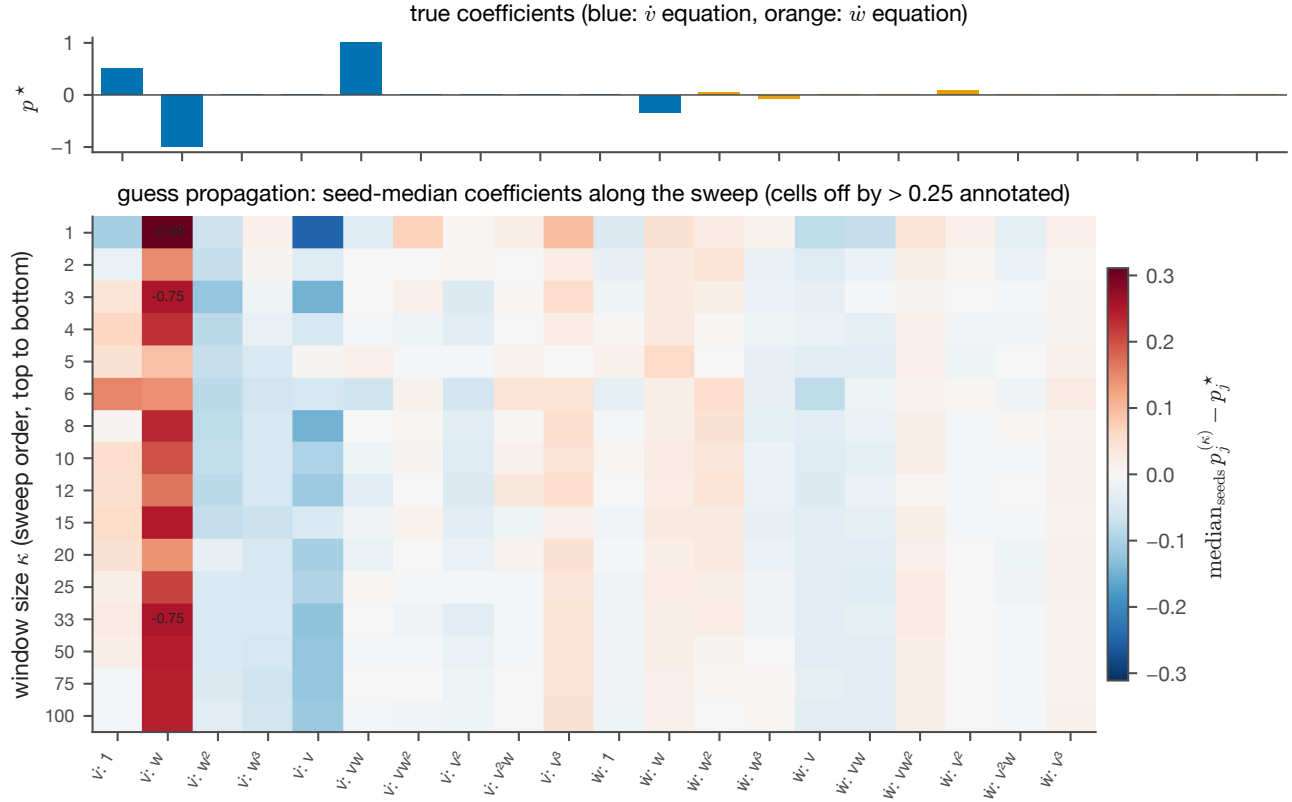
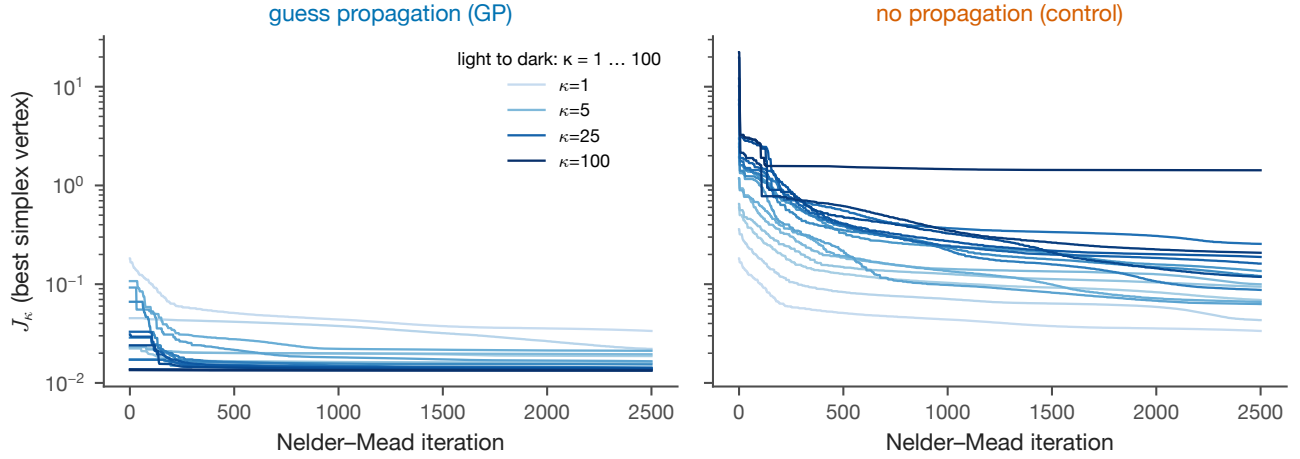
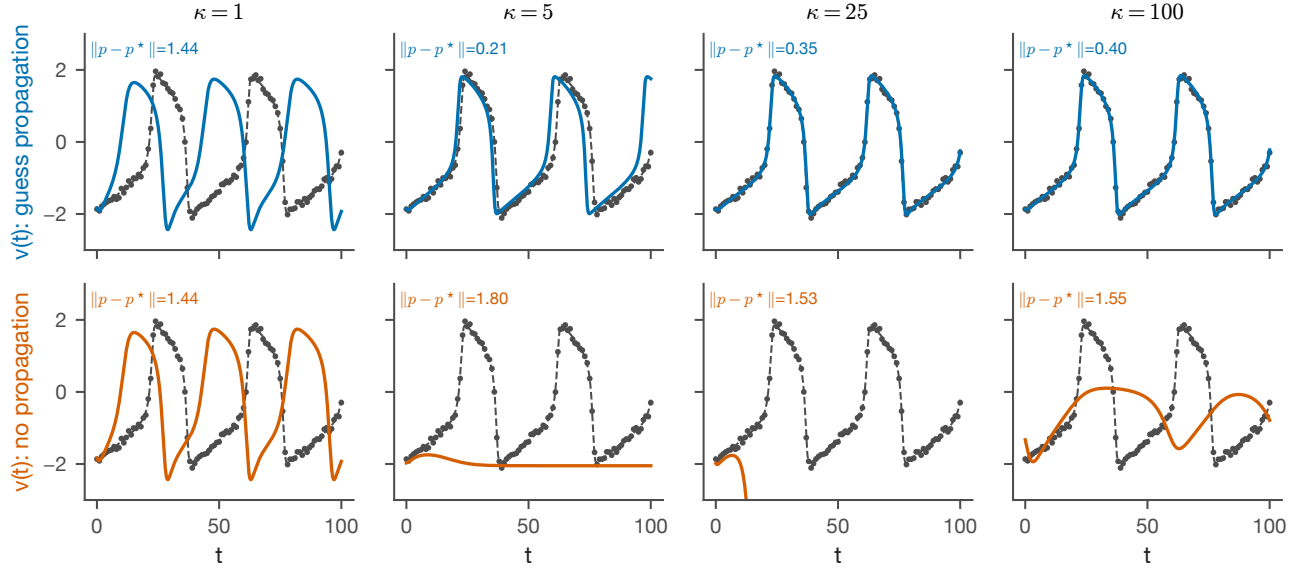


Figure 17: Which coefficients are recovered when. Top: the true coefficients (library order, $\dot{v}$ block blue, $\dot{w}$ block orange). Bottom: seed-median GP coefficient minus truth for every stage of the sweep (rows) and every coefficient (columns), diverging colour scale. The v and constant coefficients of $\dot{v}$ are right within 0.1 from $\kappa = 2$; the w coefficient of $\dot{v}$ stays ≈ 0.3 too small at every κ and the v coefficient ≈ 0.1 too small — the shrinkage of the smooth- ℓ_1 term at $\gamma = 0.05$ (cf. 24, where $\gamma = 0.01$ removes the shrinkage but loses sparsity); the slow $\dot{w}$ coefficients (≤ 0.08 in magnitude) are within 0.05. Data: `sweep_minimizers.csv`.

single-shooting simulation from the minimiser found at each window size (optimiser seed 2); grey: truth and data



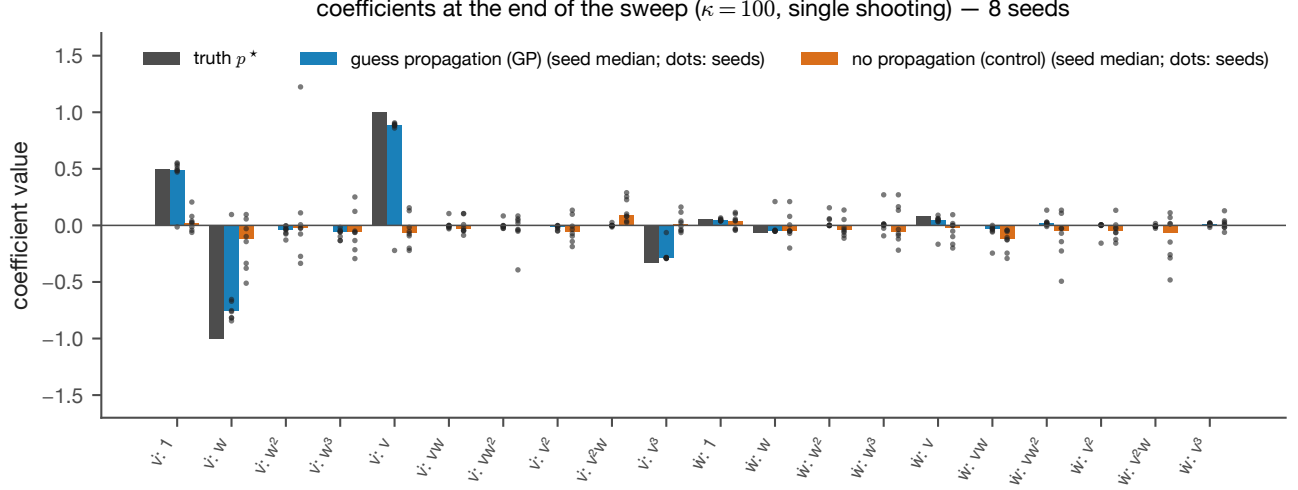


Figure 20: Coefficients at the end of the sweep ($\kappa = 100$). Truth (grey), GP seed-median (blue) and control seed-median (vermilion) with the 8 individual seeds as dots, correctly labelled in the library order (the repository’s bar-chart labels encoded the pre-refactor order — fix N1, §10). GP recovers the support (7 non-zeros, no false positive in the median) with the shrinkage noted in 17; the control’s median is near zero everywhere because its seeds scatter. Data: `sweep_minimizers.csv`.

Three GP seeds on the plateau at $\kappa = 1$, one forever. Table 2 shows 3 GP seeds with a blow-up minimiser at $\kappa = 1$ but only 1 from $\kappa = 2$ on. A random start with a positive cubic coefficient of order 0.1–0.3 blows up within one time unit ($\dot{v} \sim cv^3$, $|v| \approx 2$), so the loss is 10^3 at the start; if the whole initial Nelder–Mead simplex sits on the plateau the optimiser declares convergence after 0 iterations (all vertices equal) and returns the seed. Two of the three starts have a simplex vertex off the plateau at $\kappa = 2$ and escape; seed 1 does not, at any κ . This is a property of the *flat* penalty, not of guess propagation; the graded penalty (22, 28) removes it.

8 Part E — Variations

Table 3 lists the end-of-sweep ($\kappa = 100$) seed medians of every variation arm; each figure below shows the whole sweep.

Table 3: End-of-sweep ($\kappa = 100$) seed medians for every variation arm: $\|p - p^*\|$, recovery score, fraction of seeds on the plateau, and median wall time per stage. Arms within an experiment share seeds and schedule (SHORT for noise/sparsity/iters/substeps/seedscale, FULL for schedule/penalty/optimizer). “seedscale” arms are named `sc<scale>_<mode>_<penalty>`. Generated from `sweeps.csv`.

experiment	arm	n_seeds	$\ p - p^*\ $ ($\kappa=100$)	score	plateau frac	wall s per κ
schedule	coarse	8	0.5239	0.55	0.125	2.069
schedule	dense	8	0.3015	0.8	0.125	2.072
schedule	jump	8	0.7635	0.425	0.125	2.098
penalty	propagate_graded	8	0.274	0.825	0	2.107
penalty	reset_graded	8	1.631	0.3	0	1.88
noise	propagate $\sigma=0.0$	6	0.2313	0.8	0.1667	2.058
noise	propagate $\sigma=0.01$	6	0.3617	0.7	0.1667	2.117
noise	propagate $\sigma=0.02$	6	0.2482	0.875	0.1667	2.102
noise	propagate $\sigma=0.05$	6	0.2217	0.8	0.1667	2.087
noise	propagate $\sigma=0.1$	6	0.3135	0.625	0.1667	2.049
noise	propagate $\sigma=0.2$	6	1.881	0.225	0.1667	2.134
noise	reset $\sigma=0.0$	6	1.633	0.25	0.3333	1.845
noise	reset $\sigma=0.01$	6	1.602	0.2	0.3333	1.878
noise	reset $\sigma=0.02$	6	1.669	0.225	0.3333	1.876
noise	reset $\sigma=0.05$	6	1.649	0.25	0.3333	1.867
noise	reset $\sigma=0.1$	6	1.679	0.225	0.3333	1.847
noise	reset $\sigma=0.2$	6	1.702	0.2	0.3333	1.849
sparsity	0.0	6	0.8142	0.375	0.1667	2.175
sparsity	0.001	6	0.8433	0.475	0.1667	2.21
sparsity	0.01	6	0.8875	0.525	0.1667	2.085
sparsity	0.05	6	0.2217	0.8	0.1667	2.051
sparsity	0.2	6	0.4741	0.65	0.1667	2.033
sparsity	1.0	6	1.01	0.6	0.1667	2.003
optimizer	bfgs_propagate	8	0.2053	0.95	0.125	1.439
optimizer	bfgs_reset	8	1.648	0.225	0.625	13.55
optimizer	lbfgs_propagate	8	0.2053	0.95	0.125	102.4
optimizer	lbfgs_reset	8	1.648	0.225	0.625	15.25
iters	250	4	1.722	0.325	0.25	0.431
iters	500	4	1.426	0.2	0.25	0.8172
iters	1000	4	0.6087	0.575	0.25	1.047
iters	2500	4	0.2658	0.775	0.25	2.014
iters	5000	4	0.1986	0.9	0.25	3.867
substeps	1	4	0.4988	0.75	0	0.2064
substeps	2	4	0.3377	0.675	0.25	0.4289
substeps	5	4	0.2853	0.825	0.25	1.027
substeps	10	4	0.2658	0.775	0.25	1.991
substeps	20	4	0.2658	0.775	0.25	4.059
seedscale	sc0.01_propagate_flat	8	0.1967	0.85	0	2.064
seedscale	sc0.01_propagate_graded	8	0.1967	0.85	0	2.062
seedscale	sc0.01_reset_flat	8	1.65	0.35	0.125	1.951
seedscale	sc0.01_reset_graded	8	1.589	0.375	0	1.88
seedscale	sc0.03_propagate_flat	8	0.2857	0.775	0	2.085
seedscale	sc0.03_propagate_graded	8	0.2857	0.775	0	2.084
seedscale	sc0.03_reset_flat	8	1.586	0.475	0.25	1.875
seedscale	sc0.03_reset_graded	8	1.573	0.35	0	1.934
seedscale	sc0.1_propagate_flat	8	0.2086	0.85	0.125	2.025
seedscale	sc0.1_propagate_graded	8	0.2086	0.85	0	2.047
seedscale	sc0.1_reset_flat	8	1.649	0.25	0.25	1.888
seedscale	sc0.1_reset_graded	8	1.631	0.3	0	1.864
seedscale	sc0.3_propagate_flat	8	0.4237	0.7	0.25	2.043
seedscale	sc0.3_propagate_graded	8	0.3495	0.825	0.125	2.179
seedscale	sc0.3_reset_flat	8	2.443	0.1	0.25	1.875
seedscale	sc0.3_reset_graded	8	2.702	0.1	0.25	1.831

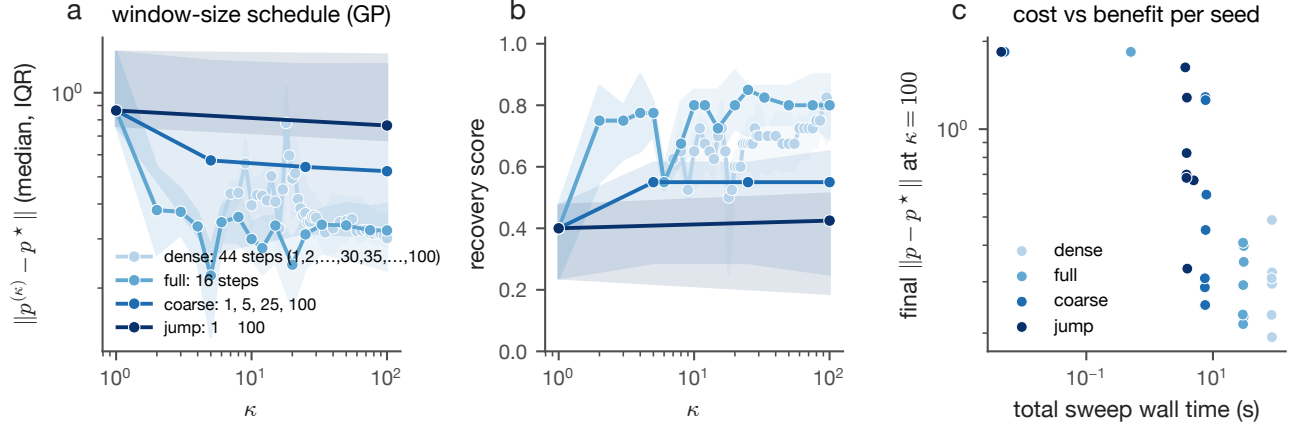


Figure 21: Window-size schedule (how many nodes to remove per step). GP with the DENSE (44 stages), FULL (16), COARSE (4) and JUMP (2: $\kappa = 1$ then 100) schedules, 8 seeds each: (a) $\|p - p^*\|$ and (b) score along the sweep (median, IQR band), (c) final error against total wall time per seed. Final medians: dense 0.30, full 0.32, coarse 0.52, jump 0.76 (scores 0.80, 0.80, 0.55, 0.43). Removing few nodes per step is what makes propagation work — the manuscript’s design rule — and the dense schedule costs $2.7\times$ the time of the full one for no gain. Data: `sweeps.csv` (`exp=schedule`, `main/propagate`).

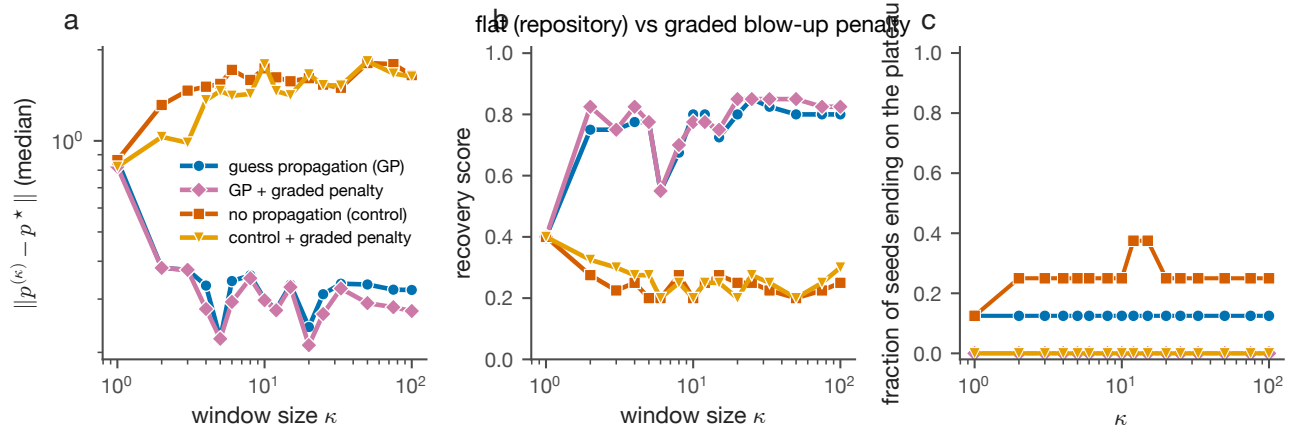


Figure 22: Flat versus graded blow-up penalty. The repository returns the constant 10^3 on blow-up; the graded variant adds a term that decreases with the index at which the blow-up occurred. GP with the graded penalty (purple) ends at $\|p - p^*\| = 0.27$, score 0.83, with *no* seed on the plateau at any stage (c), versus 0.32 / 0.80 / 12.5% for the flat penalty (blue): seed 1 is rescued. For the control the penalty changes nothing (still ≈ 1.6): without propagation the failure is the landscape, not the plateau. Data: `sweeps.csv` (`exp=penalty`, `main`).

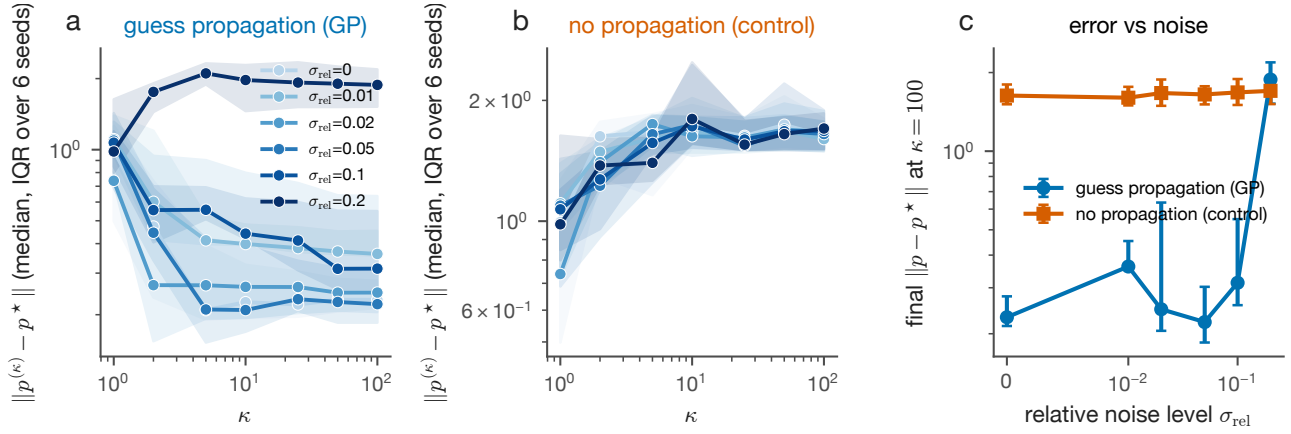


Figure 23: Observation noise. GP (a) and control (b) along the SHORT schedule for relative noise $\sigma_{\text{rel}} \in \{0, 0.01, 0.02, 0.05, 0.1, 0.2\}$ (6 seeds each; the data-noise seed is fixed so the noise realisations are scaled versions of one another); (c) final error at $\kappa = 100$. GP ends at 0.22–0.36 for $\sigma_{\text{rel}} \leq 0.1$ and fails at 0.2 (1.88); the control ends at 1.6–1.7 at *every* noise level including 0 — restarting from the seed fails because of the landscape, not the noise. Note that the GP error does not go to zero at zero noise (0.23): the remaining error is the sparsity-term shrinkage and Nelder–Mead’s finite accuracy, not noise. Data: `sweeps.csv` (`exp=noise`).

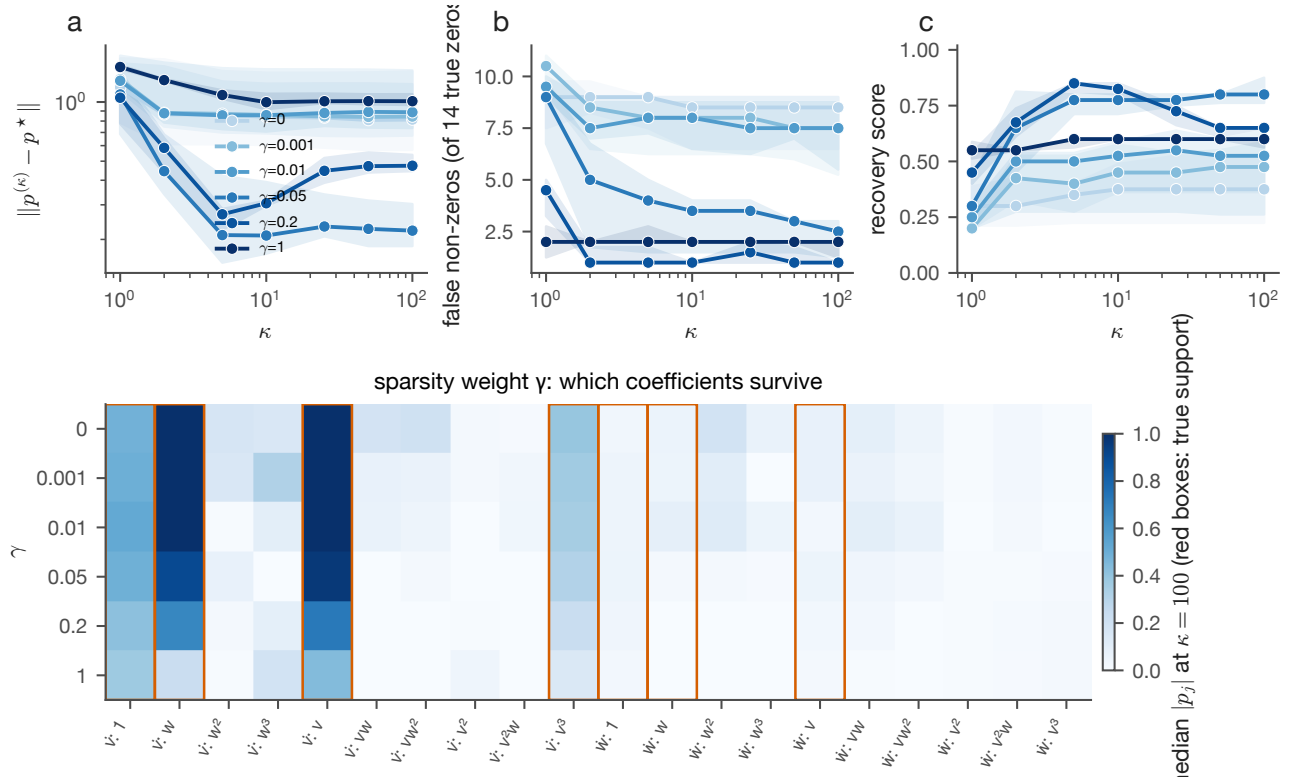


Figure 24: Sparsity weight γ . GP along the SHORT schedule for $\gamma \in \{0, 10^{-3}, 10^{-2}, 0.05, 0.2, 1\}$, 6 seeds: (a) $\|p - p^*\|$, (b) false non-zeros, (c) score; bottom: median $|p_j|$ at $\kappa = 100$ per coefficient (red boxes = true support). $\gamma = 0.05$ (the repository default) is the best of the six (0.22, score 0.80); with $\gamma \leq 0.01$ the Nelder–Mead sweep wanders into dense, wrong models (0.81–0.89, 4–6 false non-zeros), and $\gamma \geq 0.2$ over-shrinks the true coefficients (0.47, 1.01). Data: `sweeps.csv`, `sweep_minimizers.csv` (`exp=sparsity`).

optimiser: derivative-free vs AD-gradient (solid: GP, dashed: no propagation)

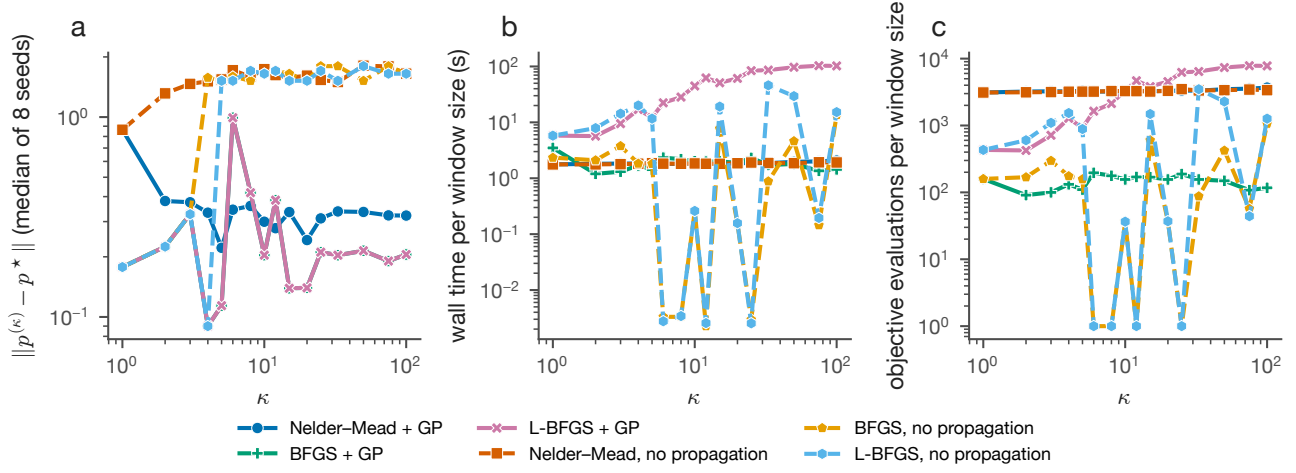


Figure 25: Optimiser: derivative-free versus AD gradient. Nelder-Mead, BFGS and L-BFGS (exact ForwardDiff gradient of the windowed loss, possible after fix F4), with (solid) and without (dashed) propagation, 8 seeds, FULL schedule: (a) median $\|p - p^*\|$, (b) wall time per stage, (c) objective evaluations per stage. BFGS+GP is the best arm (0.21, score 0.95) and needs ≈ 150 evaluations per stage instead of 3500; L-BFGS reaches the same minimiser but hits the iteration cap at large κ (100s per stage). Without propagation the gradient methods stop after one evaluation whenever the start is on the plateau (zero gradient) — 62–71 % of their stages — and otherwise converge to spurious minima. Data: `sweeps.csv` (`exp=optimizer, main`).

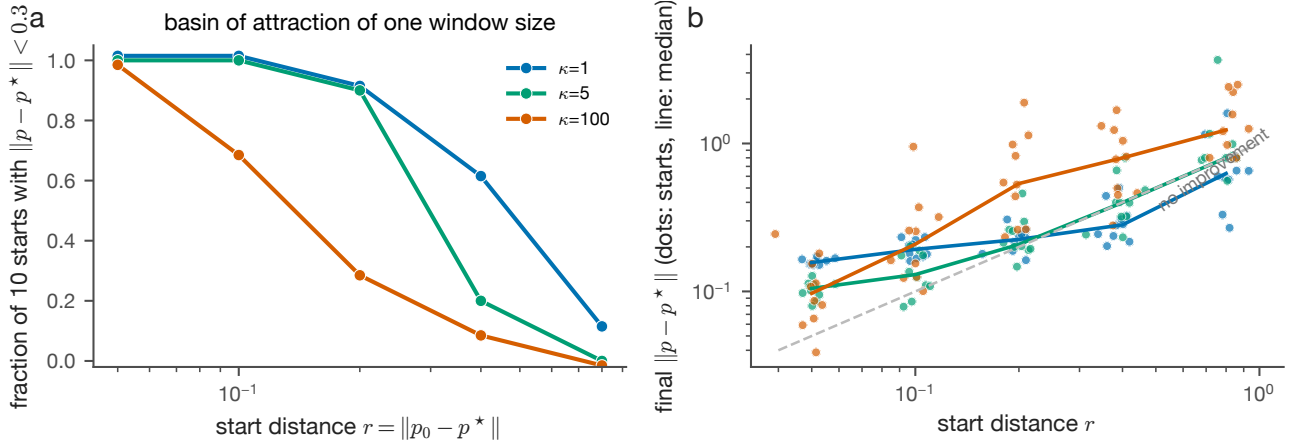


Figure 26: Basin of attraction of a single window size. One Nelder-Mead run (2500 iterations) of J_κ from $p_0 = p^* + r u$ (10 random unit directions per radius $r \in \{0.05, \dots, 0.8\}$) for $\kappa = 1, 5, 100$: (a) fraction of starts ending within 0.3 of p^* , (b) final error (dots) and median. Single shooting ($\kappa = 100$) only converges from $r \leq 0.1$; $\kappa = 5$ from $r \leq 0.2$; $\kappa = 1$ never gets below ≈ 0.15 (its noise/shrinkage floor) but improves every start up to $r = 0.4$. This is the quantitative version of 6: small windows have the wide basin, large windows have the accurate minimum. Data: `sweeps.csv` (`exp=basin`).

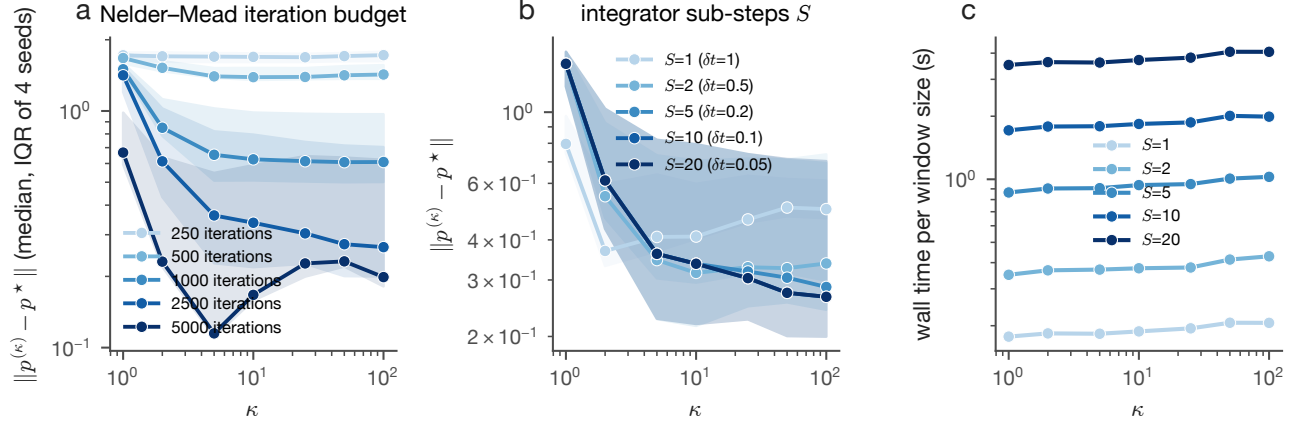


Figure 27: Iteration budget and integrator resolution. (a) GP with 250–5000 Nelder–Mead iterations per stage (4 seeds, SHORT): final medians 1.72, 1.43, 0.61, 0.27, 0.20 — fewer than 2500 iterations in 22 dimensions is not converged; (b) sub-steps $S \in \{1, 2, 5, 10, 20\}$ per data interval ($\delta t = 1/S$): $S \geq 5$ gives the same answer (0.27–0.29), $S = 1$ ($\delta t = \Delta t$) biases the fit (0.50) but never blows up; (c) wall time scales linearly with S . Data: `sweeps.csv` (`exp=iters`, `exp=substeps`).

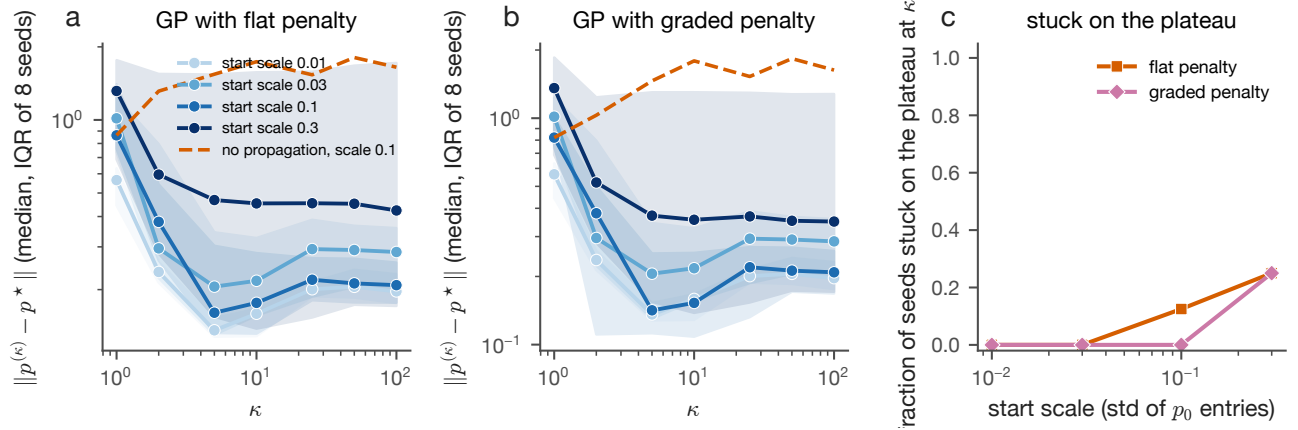


Figure 28: Start distance. GP along the SHORT schedule from $p_0 = \text{scale} \times \xi$ with scale 0.01 (the repository’s driver), 0.03, 0.1 (R001 and this report) and 0.3, 8 seeds, with the flat (a) and graded (b) penalty; the dashed reference is the control at scale 0.1; (c) fraction of starts stuck on the plateau at $\kappa = 1$. Final medians: 0.20, 0.29, 0.21, 0.42 (flat) and 0.20, 0.29, 0.21, 0.35 (graded). With the flat penalty 12.5–25% of the starts at scale ≥ 0.1 never leave the plateau; the graded penalty reduces that to 0–12.5%. Data: `sweeps.csv` (`exp=seedscale`).

9 Part F — Integrators and solvers inside the loss

Table 4: Loss evaluation at p^* , $\kappa = 5$, $\gamma = 0$, for 14 integrator settings inside the windowed loss (in-house fixed-step Tsit5 with S sub-steps; DifferentialEquations.jl solvers through the frozen mod 8 stiff branch); reference = adaptive Tsit5 at tolerance 10^{-12} . Time = median of 5 evaluations. From `solvers_eval.csv`.

method	setting	J	$ J - J_{\text{ref}} $	ms per eval
reference_diff_eq_Tsit5_adaptive	abstol=reltol=1e-12	0.0132220	0	2.296
inhouse_Tsit5_fixed	S=1	0.2818782	0.2687	0.05596
inhouse_Tsit5_fixed	S=2	0.0132233	1.262e-06	0.1089
inhouse_Tsit5_fixed	S=5	0.0132220	1.681e-10	0.2711
inhouse_Tsit5_fixed	S=10	0.0132220	1.359e-13	0.5429
inhouse_Tsit5_fixed	S=20	0.0132220	9.17e-15	1.085
inhouse_Tsit5_fixed	S=50	0.0132220	2.382e-15	2.826
diff_eq_Tsit5_adaptive	abstol=reltol=1e-4	0.0132225	5.223e-07	0.2142
diff_eq_Tsit5_adaptive	abstol=reltol=1e-8	0.0132220	2.201e-10	0.5102
diff_eq_Rosenbrock23_adaptive	abstol=reltol=1e-8	0.0132218	1.747e-07	5.689
diff_eq_Rodas5_adaptive	abstol=reltol=1e-8	0.0132220	3.545e-10	1.976
diff_eq_RadauIIA5_adaptive	abstol=reltol=1e-8	0.0132220	7.083e-09	1.327
diff_eq_ImplicitEuler_fixed	dt=0.1	0.0120178	0.001204	1.537
diff_eq_Tsit5_fixed	dt=0.1	0.0132220	1.364e-13	0.7591

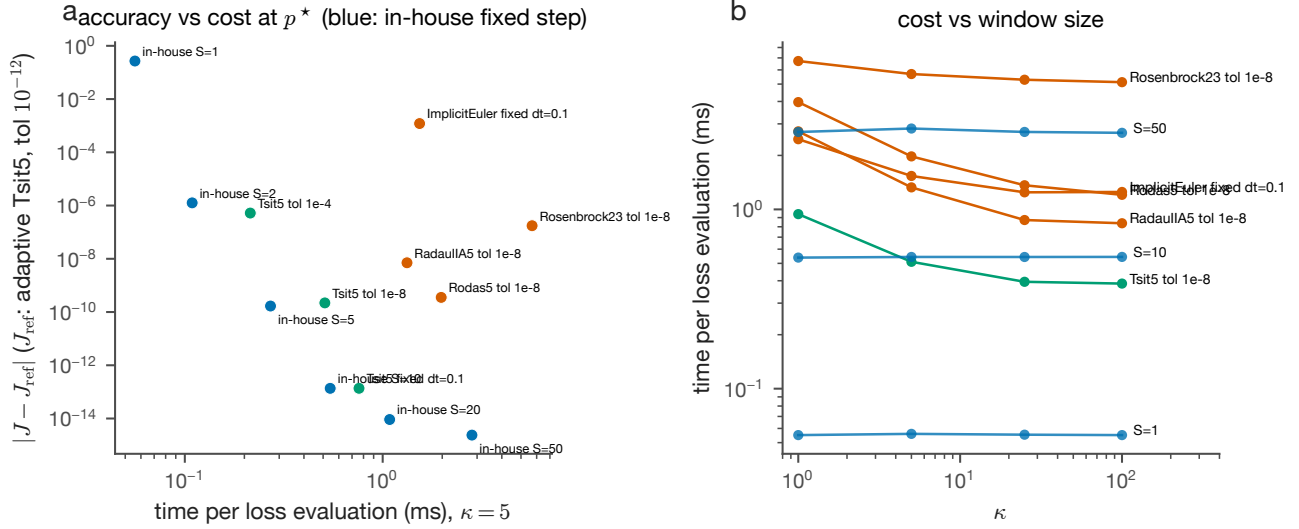


Figure 29: Accuracy versus cost of the integrator inside the loss. (a) $|J - J_{\text{ref}}|$ against the time per evaluation at $\kappa = 5$, p^* , for the 14 settings of Table 4. The in-house fixed-step Tsit5 at $S = 10$ (the default) agrees with the reference to 1.4×10^{-13} at 0.54 ms; the adaptive DifferentialEquations Tsit5 at 10^{-8} is as accurate at the same cost; implicit solvers (Rosenbrock23, Rodas5, RadauIIA5) cost 3–10 $\times$ more for no gain — the problem is not stiff; only $S = 1$ (2.7×10^{-1}) and fixed-step implicit Euler (1.2×10^{-3} , 9% of J) distort the objective. (b) Time per evaluation versus κ for a subset. Data: `solvers_eval.csv`.

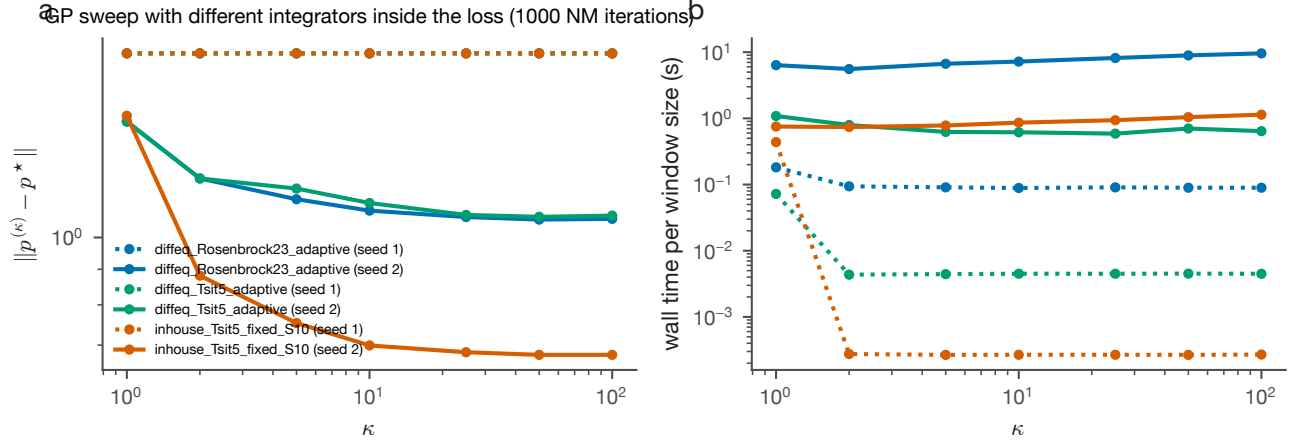


Figure 30: GP sweep with three integrators inside the loss. In-house Tsit5 ($S = 10$), adaptive DifferentialEquations Tsit5 (10^{-8}) and Rosenbrock23 (10^{-8}), SHORT schedule, 1000 Nelder–Mead iterations, seeds 1 (dotted) and 2 (solid): (a) $\|p - p^*\|$, (b) wall time per stage. Seed 1 is the plateau start and fails identically in all three (1.84 at every stage, 0 iterations); seed 2 converges with all three, the two DifferentialEquations arms agreeing exactly at $\kappa \leq 2$ and diverging afterwards through the chaotic sensitivity of a derivative-free optimiser to 10^{-10} differences in the objective (final 0.68 vs 1.06–1.08 at only 1000 iterations). Rosenbrock23 costs $8\times$ the time. Data: `solvers_sweep.csv`.

10 Part G — Defects found and fixed

Each defect is quantified by executing the frozen pre-fix source (`deps/prefix/`) against the frozen post-fix source (`deps/`), never by re-typing; `analysis/06_bugs.jl`. Table 5 summarises. The fixes are in the live tree in the same commit as this report (every changed line carries the comment R002 fix):

- **F1** (R001; mod 6, `odefun_new`): the continuation lines beginning with `+` were separate, discarded statements, so the Jacobian behind the stability/eigenvalue movies was that of the *linear truncation* of the vector field. Fixed by parenthesising the RHS. All movies under `figs/window_size_plots/stability/` are wrong and should be regenerated.
- **F2** (R001; mod 7 stiff branch of `forward_simulation_loss_windows`): the same parsing trap dropped the entire w -residual. Fixed by parenthesising. (Mod 8 already used a matrix norm and was not affected.)
- **F4** (R001; mod 7 and mod 8 drivers): `grad_fs_loss_window!` differentiated the single-shooting loss `fs_loss`, not `fs_loss_window`. Fixed; this is what makes the BFGS arms of 25 possible.
- **F5** (R001; `plotting_window_size.jl`, `animate_contourplots!`): stale call signature (missing `unstable_param` slot, keyword `silent` instead of `silent_`). Fixed to the mod 6+ signature; *not executed here* (Makie cannot load in the sandbox), so this fix is a reading, not a measurement.
- **F6** (R001; mod 8 driver): the `@threads` loop wrote the shared `x0`, `results_inner` and a `Dict`. Fixed with per-run copies and a lock; not a figure.
- **N1 (new)** mod 8 driver: the sweep called the pre-refactor `odefun` (order 1, $v, w, \dots$) while the data, `fhn_p` and the polish used the library order; the bar-chart labels `trmstr` encoded the old order and `plotSolution!` was called with an undefined name `trm`. The truth vector read in the wrong order is 2.50 away from itself (34a) and its cost at $\kappa = 1$ is 0.68 instead of 0.018: the committed sweep could not have recovered the model. Fixed (sweep and plots use `odefun_generic`; labels generated from `multiindex_mapping`).
- **N2 (new)** mod 8, `forward_simulation_loss` (explicit branch, used by the final BFGS polish): read the *global data* instead of its `data_` argument, so the function ignored the data it was given (34b). Fixed. This is the shadowing R001 asked to check when the refactor landed.
- **N3 (new, behavioural)** the committed driver had guess propagation *disabled* (`copyto!(x0, x0_)` after every stage). Replaced by an explicit `guess_mode` switch (`:propagate` default, `:best`, `:reset`).

- Minor: undefined `y0` default in `custom_integrator`; wrong bound in the window-size warning.

R001’s bug-presence gates G3, G4, G5 and G7 are retired by this report: R002’s gates G5–G9 assert the defects are present in `deps/prefix/` and absent in `deps/`.

Table 5: Defect quantification (pre-fix vs post-fix frozen sources). From `bug_summary.json` and the `bug_*.csv` files.

finding	measure	value
F2 (R001): mod 7 stiff branch drops the w -residual	share of $J(p^*)$ missing, $\kappa=1/10/25/100$	11.5 % / 9.3 % / 17.0 % / 11.2 % / 11.9 %
F1 (R001): mod 6 <code>odefun_new</code> = linear truncation	$\partial\dot{v}/\partial v$ at $u=(1.5,0.4)$: pre-fix vs fixed	1 vs -1.25
F4 (R001): sweep gradient = single-shooting gradient	$\cos\angle(g\kappa, g100)$ at $\kappa=1 / 5$	0.180 / 0.989
NEW: mod 8 driver mixed monomial orders	spurious $\ p - p^*\ $ between orderings	2.501
NEW: mod 8 forward_simulation_loss ignores <code>data_</code>	J identical for 3 datasets pre-fix / differs post-fix	True / True

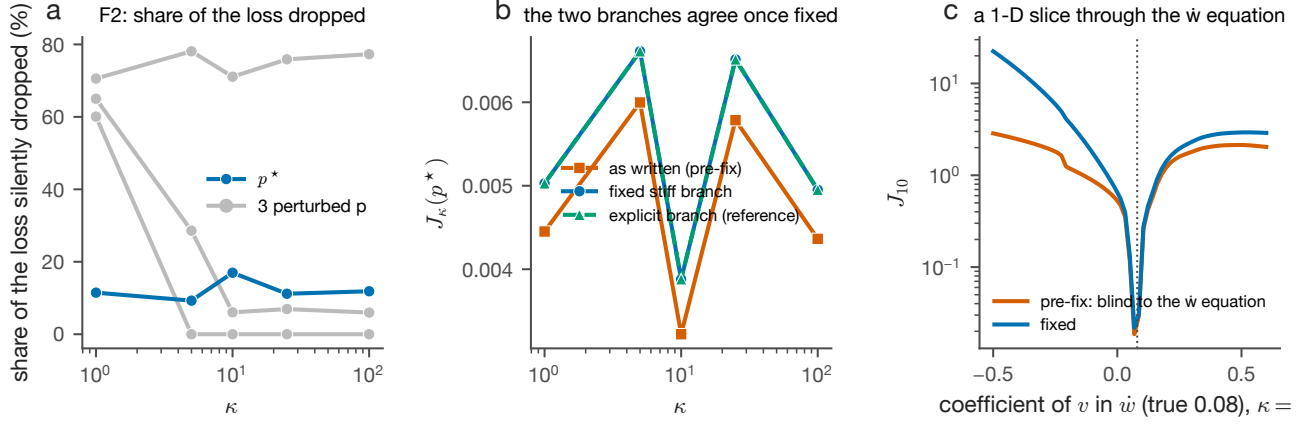


Figure 31: F2: the mod 7 stiff branch silently discarded the w -residual. (a) Share of the loss missing, $(J_{\text{fixed}} - J_{\text{prefix}})/J_{\text{fixed}}$, at p^* (blue: 11.5, 9.3, 17.0, 11.2, 11.9% at $\kappa = 1, 5, 10, 25, 100$, $\gamma = 0$) and for three perturbed parameter vectors (grey; up to 78% — away from the optimum the w trajectory is badly wrong and the dropped term dominates; rows at 0 are blow-ups, where both versions return 10^3). (b) The three implementations at p^* : pre-fix stiff branch, fixed stiff branch and the explicit in-house branch agree to 3×10^{-10} once fixed. (c) A 1-D slice through the v coefficient of the $\dot{w}$ equation at $\kappa = 10$: the fixed objective varies by a factor 10^3 across the slice, the pre-fix one barely moves — the stiff branch was blind to the $\dot{w}$ equation, so every contour/surface movie made with `stiff_solver=true` shows the wrong landscape. Data: `bug_F2_w_residual.csv`, `bug_F2_slice.csv`.

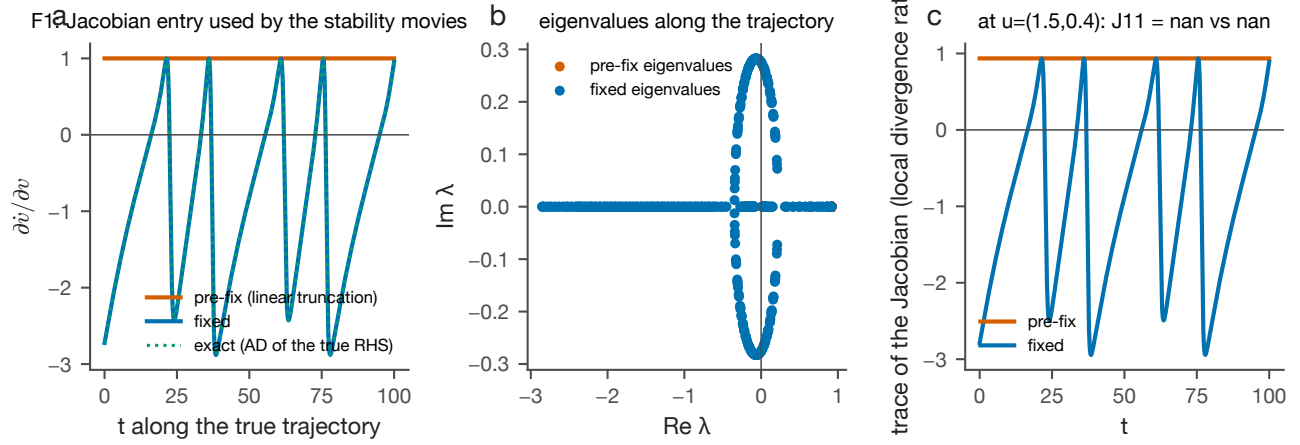


Figure 32: F1: the stability movies used a linear truncation of the vector field. Along the true FHN trajectory (401 points): (a) $\partial \dot{v}/\partial v$ from the pre-fix `odefun_new` (vermilion, constant +1), the fixed one (blue) and the exact Jacobian of the true RHS (green dotted, identical to fixed to 0.0); at $u = (1.5, 0.4)$ the entry is +1.0 instead of -1.25 — the wrong sign. (b) Jacobian eigenvalues along the orbit, pre-fix (a fixed pair) versus fixed (moving with the state, crossing into the stable half-plane on the slow branches). (c) The trace (local divergence rate), constant and positive pre-fix, mostly negative after the fix. Data: `bug_F1_jacobian.csv`, `bug_F1_meta.json`.

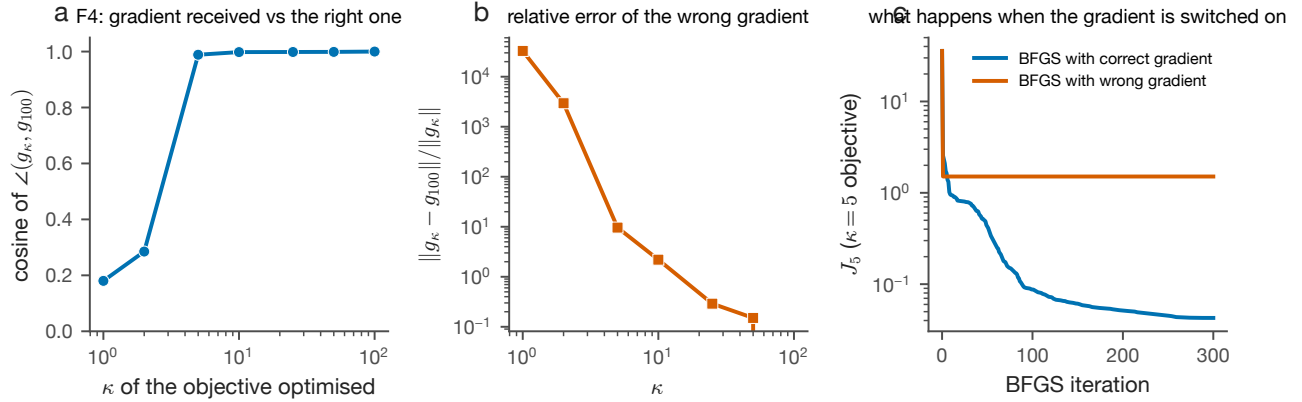


Figure 33: F4: the sweep's gradient was the gradient of the wrong function. At $z = [y_0; p^* + 0.05 \xi]$: (a) cosine between the gradient of the κ -windowed objective and the single-shooting gradient the pre-fix code returned — 0.18 at $\kappa = 1$ (80°), 0.99 at $\kappa = 5$, 1 at $\kappa = 100$; (b) the relative error of the wrong gradient, 3×10^4 at $\kappa = 1$ (the single-shooting gradient is huge because that trajectory has diverged). (c) 300 BFGS iterations on the $\kappa = 5$ objective from the same start with the correct gradient (J_5 : 35.8 $\rightarrow$ 0.043) and with the wrong one (stalls at 1.51 from iteration 19, 23 708 line-search evaluations). The defect was harmless only because the sweep used Nelder–Mead, which ignores the gradient. Note: this start ($\|p - p^*\| = 0.23$) has a large cost because the 0.05 perturbation hits the cubic term; the correct-gradient run lowers J_5 but ends at $\|p - p^*\| = 3.96$ (a different, wrong minimum) — the panel demonstrates the gradient, not recovery. Data: `bug_F4_gradient.csv`, `bug_F4_bfgs.csv`.

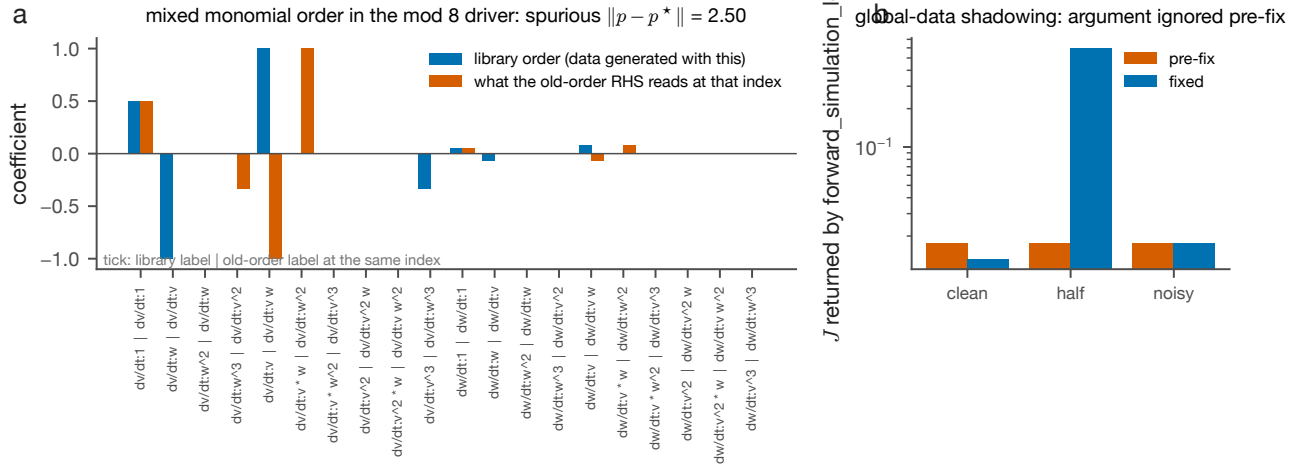


Figure 34: New defects in the committed mod 8 driver. (a) N1, mixed monomial order: the true coefficients in the library order the data were generated with (blue) and what the old-order RHS used by the sweep reads at the same index (vermilion); tick labels give both names. The two readings of the same vector are 2.50 apart (larger than $\|p^*\| = 1.54$), and the true vector evaluated through the old-order RHS costs 0.68 ($\kappa = 1$), 2.06 ($\kappa = 5$), 7.83 ($\kappa = 100$) instead of 0.018–0.021. (b) N2, global-data shadowing: `forward_simulation_loss` called with three different data matrices (noisy, clean, states halved) returns the same 0.0176 pre-fix and 0.0176 / 0.0133 / 0.592 after the fix. Data: `bug_ordering.csv`, `bug_ordering_meta.json`, `bug_shadowing.csv`.

11 Part H — Other systems and closing the loop with the theory

The core is dimension- and degree-agnostic; two further systems test it (Table 6). Lotka–Volterra: $\dot{x} = x - 0.5xy$, $\dot{y} = -0.8y + 0.3xy$, quadratic library (6 monomials, 12 coefficients), $N = 101$, $\Delta t = 0.25$, 5% noise, Nelder–Mead. Lorenz: $\sigma = 10$, $\rho = 28$, $\beta = 8/3$, quadratic library (10 monomials, 30 coefficients), $N = 101$, $\Delta t = 0.02$ ($t \in [2, 4]$ of a trajectory from $(-8, 7, 27)$), 2% noise, start scale 0.5; BFGS (2500-iteration cap) and Nelder–Mead (4000). Same SHORT schedule and $\gamma = 0.05$.

Table 6: Other systems at $\kappa = 100$: seed medians of $\|p - p^*\|$ and score, and the fraction of seeds on the plateau. From `other_sweeps.csv`.

system	optimizer	arm	n_seeds	$\ p - p^*\ $ ($\kappa=100$)	score	plateau frac
lorenz	bfgs	propagate	4	3.852	0.8333	0
lorenz	bfgs	reset	4	31.95	0.03333	1
lorenz	nm	propagate	4	18.11	0.2333	0
lorenz	nm	reset	4	31.95	0.03333	1
lv	nm	propagate	6	0.29	0.8333	0
lv	nm	reset	6	1.408	0.2917	0.8333

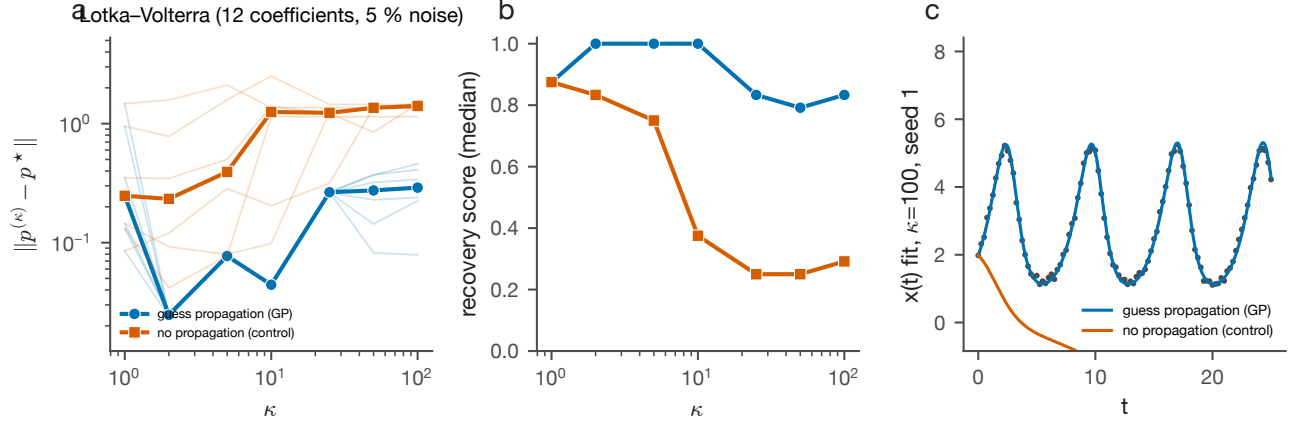


Figure 35: Lotka–Volterra. (a) $\|p - p^*\|$ along the sweep (thin: 6 seeds, thick: median), (b) score, (c) the $x(t)$ fit at $\kappa = 100$ for seed 1. GP ends at 0.29 (score 0.83; one seed reaches 0.025 with score 1.0 at $\kappa = 2$ –10, i.e. exact recovery), the control at 1.41 with 5 of 6 seeds on the plateau; GP is better in every seed. Data: `other_sweeps.csv`, `other_fits.csv` (`system=lv`).

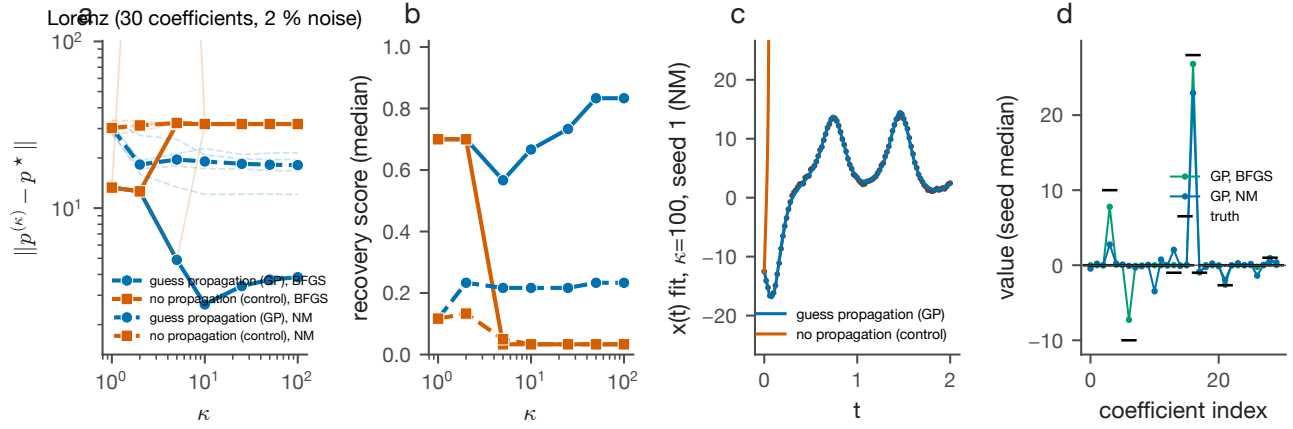


Figure 36: Lorenz (chaotic, 30 coefficients). (a) $\|p - p^*\|$ along the sweep for BFGS (solid) and Nelder–Mead (dashed), GP (blue) and control (vermillion), 4 seeds; (b) score; (c) the $x(t)$ fit at $\kappa = 100$, seed 1; (d) the seed-median GP coefficients versus truth. GP+BFGS recovers Lorenz ($\|p - p^*\| = 3.9$ on coefficients of size up to 28, score 0.83, no blow-up); GP+Nelder–Mead never converges within 4000 iterations in 33 dimensions (18.1, score 0.23) — a derivative-free sweep does not scale; the control blows up in 4 of 4 seeds from $\kappa = 5$ on with either optimiser. Data: `other_sweeps.csv`, `other_minimizers.csv`, `other_fits.csv` (`system=lorenz`).

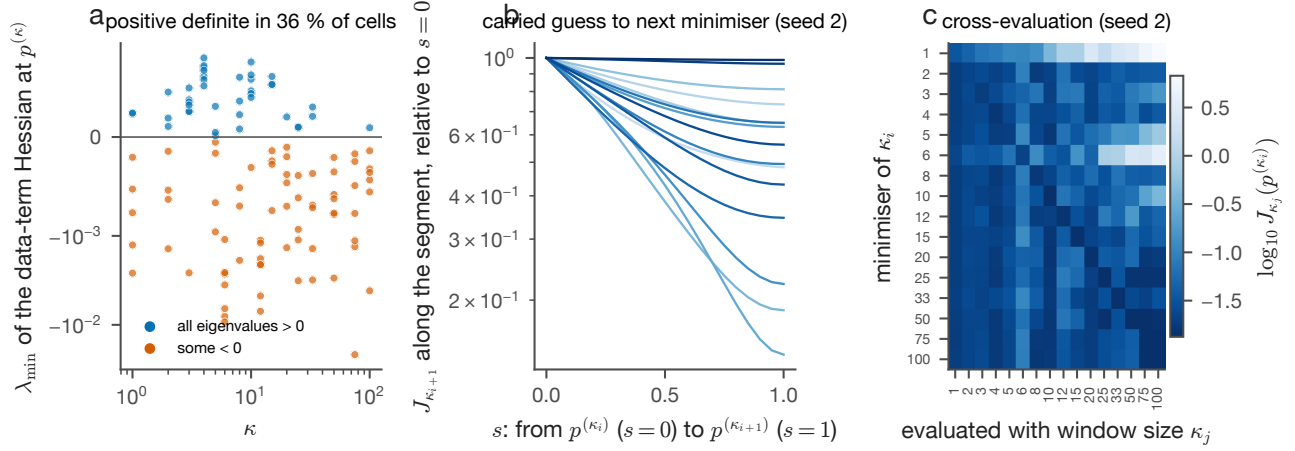


Figure 37: The theory’s premises checked where they are assumed. (a) Smallest eigenvalue of the data-term Hessian at the GP minimiser $p^{(\kappa)}$ returned by Nelder–Mead at every stage, 7 seeds (the plateau seed excluded; symlog scale; blue = all 20 eigenvalues positive, vermilion = at least one negative). Where the manuscript assumes local strong convexity — at the minimiser, not at p^* — the returned point is positive definite in only 36 % of the (seed, κ) cells, with on average 0.8 of 20 eigenvalues slightly negative elsewhere ($|\lambda_{\min}| \lesssim 10^{-3}$ against $\lambda_{\max}$ of 10^{-6}): Nelder–Mead stops short of a true stationary point in 22 dimensions (cf. 27a), so the premise is *not* verified at the returned iterates; the BFGS iterates of 25 would be the right place to test it. (b) The cost of the *next* stage along the straight segment from $p^{(\kappa_i)}$ to $p^{(\kappa_{i+1})}$ (seed 2), relative to its value at the carried guess: it decreases monotonically along every segment, i.e. the carried guess lies inside the basin of the next minimiser — the mechanism the node-removal theorem describes. (c) Cross-evaluation (seed 2): the minimiser of κ_i evaluated at every other κ_j (log colour); the near-diagonal structure shows that a minimiser is good only for nearby window sizes, which is why the schedule must be gradual (21). Data: `post_hessian_at_minimizers.csv`, `post_path_between_minimizers.csv`, `post_cross_evaluation.csv`.

12 Gates

`analysis/verify_R002.py` reads only this folder’s own result files and writes `analysis/results/gates_summary.json`. **Status at the final build: 18 of 18 gates pass .** Gates G5–G9 are bug-presence/absence gates (pre-fix present, post-fix absent); they are expected to keep passing as long as `deps/prefix/` is the 44abf4a snapshot. All other gates are ordinary correctness or completeness gates.

gate	asserts
G1	all 12 frozen files hash to <code>deps/MANIFEST.md</code>
G2	<code>ms_loss</code> = frozen mod 8 loss, relative difference $< 10^{-10}$ (42 evaluations)
G3	dataset = frozen mod 8 data block, $< 10^{-12}$
G4	<code>make_rhs</code> = <code>odefun_poly!</code> and = old-order <code>odefun</code> under the permutation, $< 10^{-12}$
G5	F1 present pre-fix ($\partial\dot{v}/\partial v = +1.0$), absent post-fix (-1.25 = exact, max deviation 0)
G6	F2 present pre-fix (5–25 % of $J(p^*)$ missing at every κ), fixed stiff = explicit to 10^{-8}
G7	F4 present pre-fix: cosine < 0.5 at $\kappa = 1$ and > 0.999 at $\kappa = 100$
G8	N1 present pre-fix: spurious $\ p - p^*\ > 1$; old-order cost of $p^* > 10\times$ the correct one
G9	N2 present pre-fix (identical J for 3 datasets), absent post-fix
G10	main sweep complete: 3 arms $\times$ 8 seeds $\times$ 16 stages = 384 rows
G11	GP tied with control at $\kappa = 1$ in every seed and strictly better in every non-plateau seed at every $\kappa \geq 2$
G12	paired GP – control CI at $\kappa = 100$ excludes 0
G13	every control row on the plateau returns exactly the seed vector
G14	in-house $S = 10$ loss within 10^{-8} of the adaptive reference at $\kappa \in \{1, 5, 25, 100\}$
G15	other systems: GP better than control in every seed (LV, Lorenz BFGS, Lorenz NM)
G16	every planned sweep job realised
G17	every figure referenced by <code>report.tex</code> exists; results ledger written
G18	Table 2’s medians reproduce from <code>sweeps.csv</code>

13 Limitations and honest notes

- **Seeds.** 8 optimiser seeds (6 for noise/sparsity, 4 for iterations/sub-steps/Lorenz, 10 starts per radius for the basin experiment) is enough for paired CIs that exclude zero, not for tail statistics; one seed in eight starts on the plateau and is kept in every median.
- **Nelder–Mead budget.** At 2500 iterations the sweep is converged for FHN (27a) but not for Lorenz (36); the BFGS arms are the recommendation, and their wall times are machine-dependent.
- **The Lipschitz bounds are loose** by orders of magnitude on this attracting system (8–10); the figures support the theory’s *ordering* of single vs multiple shooting, not the size of the constants.
- **Not gated:** F5 (Makie) and F6 (a race) are readings; the concept figures 3 and 7 are schematics and say so.
- **Timing columns** are wall-clock on a shared 16-core machine with up to 16 concurrent processes and are only comparative.
- **Planned vs realised.** 522 planned sweep jobs; the batch was re-sharded twice (resume logic in `04_sweeps.jl`) after two shards stalled on slow L-BFGS jobs; duplicates from deterministic re-runs are removed at merge time (identical results, different wall time). The first-batch sparsity jobs (arm name without γ) were superseded by γ -named arms and are dropped at merge.

Reproduction

See `REPRODUCE.md` for the pinned environment and the ordered commands (Julia stages 00–09, then the Python inventory, tables, figures, animation, gates, number manifest, and `pdflatex` twice). `report.md` is a full-content Markdown rendition of this document generated from the same sources.